PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

MEDICARE PLUS – HOSPITAL MANAGEMENT SYSTEM

AN INTERNSHIP REPORT

Submitted by

HARSHITH V - 20221CSE0635

Under the guidance of,

Mr. H Senthil Kumar

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report **"MediCare Plus – Hospital Management System"** is a bonafide work of **Harshith V (20221CSE0635),** who has successfully carried out the internship work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING during 2025-2026.

Mr. H Senthil Kumar
Internship Guide
PSCS
Presidency University

Mr. Sakthi S
Program Internship Coordinator
PSCS
Presidency University

Dr. Md Ziaur Rahaman
School Internship Coordinator
PSCS
Presidency University

Dr. H B Asif Mohammed
Head of the Department
PSCS
Presidency University

Dr. Duraipandian N
Dean
PSCS / PSIS
Presidency University

Name and Signature of the Examiners

Sl. No	Name	Signature	Date
1	Mr. Jerrin Joe Francis		
2	Ms. Nayana R		

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I am a student of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING, at Presidency University, Bengaluru, named **HARSHITH V**, hereby declare that the internship work titled **"MediCare Plus – Hospital Management System"** has been independently carried out by me and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER SCIENCE ENGINEERING during the academic year of 2025-2026. Further, the matter embodied in the internship has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

HARSHITH V USN: 20221CSE0635 Signature:

PLACE: BENGALURU

DATE: 23 - April 2026

INTERNSHIP COMPLETION CERTIFICATE

रेल पहिया कारखाना
RAIL WHEEL FACTORY
भारत सरकार / Govt. of India
रेल मंत्रालय / Ministry of Railways

महा प्रबंधक का कार्यालय,
कार्मिक विभाग
यलहंका, बैंगलूरु - 560 064
General Manager's Office,
Personnel Department
Yelahanka, Bangalore - 560 064

NO.RWF/NG-16/348/Pt III

DATE: 21.04.2026

CERTIFICATE

This is to certify that **Mr. HARSHITH V, (20221CSE0635)**, B.Tech (Computer Science & Engineering) Student of Presidency University, has underwent Internship on **"Medicare Plus-Hospital Management System" at Rail Wheel Factory, Yelahanka, Bangalore** for a period of 03 months i.e., 03rd January 2026 to 10th April 2026.

The student has shown keen interest in learning during the period he was with us for the Internship. We found him Enthusiastic, Diligent, Hard Working and Creative.

We wish a bright future and success in all his endeavors.

(K.S.KIRAN)
SENIOR PERSONNEL OFFICER-I

Tel : +91-80-28072402 Fax : +91-80-28460306

ACKNOWLEDGEMENTS

For completing this internship work, I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. I extend my gratitude to our beloved **Chancellor, Vice Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the internship.

I would like to sincerely thank my internal guide **Mr. H Senthil Kumar, Assistant Professor,** Presidency School of Computer Science and Engineering, Presidency University, for his moral support, motivation, timely guidance and encouragement provided to me during the period of internship work.

I am also thankful to **Dr. H B Asif Mohammed, Associate Professor, Head of the Department, Presidency School of Computer Science and Engineering** Presidency University, for his mentorship and encouragement.

I express my cordial thanks to **Dr. Duraipandian N**, Dean PSCS & PSIS and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my internship work.

I am grateful to **Dr. Md Ziaur Rahaman,** Internship Coordinator and **Mr. Sakthi S,** Program Internship Coordinator, Presidency School of Computer Science and Engineering, or facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

I am also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

Harshith V

ABSTRACT

The rapid advancement of digital technologies has significantly transformed the healthcare sector, necessitating the adoption of integrated and intelligent systems for managing complex hospital operations. Traditional hospital management approaches, which often depend on manual record-keeping or fragmented software solutions, suffer from critical limitations such as data inconsistency, redundancy, lack of interoperability, and delayed access to patient information. These inefficiencies hinder administrative workflows and negatively impact the quality and timeliness of patient care.

To address these challenges, this project introduces Medicare Plus, a comprehensive and scalable web-based Hospital Management System (HMS) designed to unify and streamline core healthcare operations within a centralized digital platform. The system was developed during an internship at Rail Wheel Factory (RWF), Yelahanka, Bengaluru, a Government of India enterprise under the Ministry of Railways, where the IT infrastructure and mentorship supported practical implementation. Medicare Plus leverages modern web technologies, utilizing Python Flask for backend development and Microsoft SQL Server for secure and structured data management. Communication between system components is facilitated through RESTful APIs, ensuring modularity, scalability, and seamless integration.

Medicare Plus is designed with a multi-role architecture, supporting key stakeholders such as Admin, Doctor, Patient, Receptionist, Pharmacist, and Radiologist. Each role is equipped with a dedicated dashboard tailored to their functional requirements, ensuring intuitive user experience and operational efficiency. Secure access control is implemented using JSON Web Token (JWT)-based authentication, enabling role-based authorization and safeguarding sensitive medical data.

The system incorporates essential functionalities such as patient management, appointment scheduling, laboratory test handling, digital prescriptions, and chronic disease monitoring. It also supports file and document management for storing medical reports, along with analytics capabilities for data-driven decision-making.

By integrating these functionalities into a single platform, Medicare Plus enhances operational efficiency by reducing paperwork, minimizing data duplication, and enabling real-time access to critical medical information. The system improves coordination among healthcare professionals, ensures faster service delivery, and enhances patient engagement, while its secure and scalable design makes it suitable for deployment across hospitals of varying sizes.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
RWF	Rail Wheel Factory
HMS	Hospital Management System
CRUD	Create, Read, Update, Delete
REST	Representational State Transfer
CSE	Computer Science and Engineering
CSS	Cascading Style Sheets
DB	Database
JWT	JSON Web Token
RBAC	Role-Based Access Control
PHI	Protected Health Information
SQL	Structured Query Language
RDBMS	Relational Database Management System
SPA	Single Page Application
UI	User Interface
UX	User Experience
CORS	Cross-Origin Resource Sharing
WSGI	Web Server Gateway Interface
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
SSL/TLS	Secure Sockets Layer / Transport Layer Security
SDLC	Software Development Life Cycle
UAT	User Acceptance Testing
ACID	Atomicity, Consistency, Isolation, Durability
EHR	Electronic Health Records
HIPAA	Health Insurance Portability and Accountability Act

GDPR	General Data Protection Regulation
HL7	Health Level Seven (standards organization)
IoT	Internet of Things
AI	Artificial Intelligence
ML	Machine Learning

Chapter 1

INTRODUCTION

1.1 Background

The healthcare industry is currently undergoing a massive paradigm shift, transitioning from legacy, paper-based administrative frameworks to highly advanced electronic healthcare solutions. Hospitals, clinics, and healthcare institutions are inherently data-intensive environments. Every single day, they generate, process, and store vast volumes of highly sensitive data, including complex patient medical histories, intricate diagnostic reports, pharmaceutical prescriptions, billing information, and dynamic appointment schedules. The efficient, secure, and rapid management of this data is not merely an administrative requirement; it is a critical component of delivering timely and accurate medical services.

However, a surprising number of healthcare facilities—particularly mid-sized clinics and regional hospitals—still rely heavily on manual record-keeping or fragmented, partially digitized systems. These older systems typically operate in silos; for example, the pharmacy software cannot communicate with the radiology department or the doctor's primary diagnostic tool. Such fragmented approaches create severe bottlenecks. Retrieving a patient's medical history during an emergency becomes a dangerously time-consuming task. Furthermore, the lack of an integrated system frequently leads to duplicated diagnostic tests, dangerous miscommunications regarding medication, and severe administrative delays that frustrate both staff and patients.

The escalating demand for superior healthcare delivery, combined with rapid advancements in full-stack web technologies, has catalyzed the evolution of modern Hospital Management Systems (HMS). In this context, Medicare Plus has been conceptualized and designed as a state-of-the-art, fully integrated solution. It directly addresses the limitations of traditional, disjointed systems by providing a cohesive digital environment that supports optimal healthcare management and continuity of care.

The internship was conducted at Rail Wheel Factory (RWF), Yelahanka, Bengaluru, which is a prestigious Government of India enterprise operating under the Ministry of Railways. Established as one of the key production units of Indian Railways, RWF plays a vital role in

the manufacturing and supply of railway wheels, axles, and wheel sets used in rolling stock across the country. The organization is known for its advanced manufacturing facilities, precision engineering practices, and strict quality control standards, which ensure the reliability and safety of railway operations. With its significant contribution to the railway infrastructure, RWF supports the efficient functioning and expansion of one of the largest rail networks in the world.

Although the core operations of the organization are industrial and manufacturing-oriented, RWF has also embraced the importance of information technology in modern organizational management. Its Management Information Service Centre (MISC) and IT division provide a strong technological backbone that supports digital initiatives, automation processes, and system optimization. During the internship, this IT ecosystem offered the necessary infrastructure, technical resources, and professional mentorship required for undertaking a software development project. The guidance provided by experienced professionals helped in understanding real-world project workflows, system design approaches, and implementation strategies.

The project carried out during the internship focused on addressing administrative challenges through digital solutions, specifically in the domain of hospital management. Despite being a manufacturing unit, the organization's support for such projects reflects its commitment to fostering innovation, interdisciplinary learning, and digital transformation. This aligns with the broader vision of Indian public sector enterprises to adopt modern technologies and improve operational efficiency across various domains.

Furthermore, the internship experience at RWF provided exposure to a structured and disciplined work environment, where emphasis is placed on teamwork, accountability, and continuous improvement. It enabled the application of theoretical knowledge to practical scenarios, bridging the gap between academic learning and industry requirements. Overall, the organization not only facilitated the successful completion of the project but also contributed significantly to professional development by providing valuable insights into industrial practices and emerging technological trends.

1.2 Statistics and Context

The increasing complexity, regulatory requirements, and sheer scale of modern healthcare operations highlight the urgent, non-negotiable need for digital transformation. Several contextual factors drive this urgency:

- High Volume and Velocity of Patient Data: Modern hospitals handle thousands of interactions daily, ranging from outpatient consultations and inpatient admissions to laboratory workflows and pharmacy dispensing. Managing these massive datasets manually or through primitive software is inefficient and highly susceptible to human error, which in healthcare can have fatal consequences.
- Inefficiencies in Manual Systems: Manual and semi-digital systems inevitably lead to data duplication, misplaced physical records, and agonizing delays in retrieving critical clinical information. When a physician cannot immediately access a patient's allergy history or previous lab results, clinical decision-making is impaired, directly threatening patient safety.
- The Post-COVID-19 Healthcare Landscape: The global pandemic acted as a powerful catalyst for the adoption of digital healthcare technologies. It exposed the fragile nature of manual hospital administration and sparked a massive surge in demand for Electronic Health Records (EHR), touchless workflows, and integrated systems that can rapidly scale during public health crises.
- The Imperative of Real-Time Access: The "Golden Hour" in medical emergencies dictates that doctors need instantaneous access to life-saving data. Healthcare providers now demand systems that offer real-time synchronization across all devices and departments, ensuring that a scan uploaded by a radiologist is immediately viewable by the attending physician.

1.3 Existing Technologies and Their Limitations

While various digital tools exist in the current market, they often fail to strike the right balance between functionality, usability, and cost-effectiveness.

Commercial Hospital Management Systems (Enterprise Solutions) Large-scale commercial HMS platforms are utilized by massive hospital networks. While they offer deep feature sets, they suffer from significant drawbacks:

- Prohibitive Costs: The licensing, implementation, and training costs are often entirely out of reach for small to medium-sized healthcare facilities.
- Vendor Lock-In and Rigidity: These monolithic systems are notoriously difficult to customize. Hospitals are forced to adapt their workflows to the software, rather than the software adapting to the hospital.
- Bloated Interfaces: They often include hundreds of unnecessary features that

overwhelm users and steepen the learning curve for non-technical medical staff.

Basic Database Tools Some clinics attempt to use standalone, localized database tools (like Microsoft Access or basic localized SQL setups) to digitize records.

- Lack of Interoperability: These tools do not natively communicate with other hospital departments.
- Poor User Experience: They lack modern, intuitive web interfaces, requiring staff to interact with clunky, outdated UI forms.
- Limited Analytics: They cannot generate the complex, real-time visual analytics required for modern hospital administration.

Spreadsheet-Based Systems In rural or underfunded clinics, spreadsheets (Excel, Google Sheets) remain the primary tool for scheduling and record-keeping.

- Zero Scalability and Integrity: Spreadsheets lack relational data integrity. A typo in a patient's ID can sever their entire medical history.
- Security and Compliance Risks: Spreadsheets cannot enforce strict Role-Based Access Control (RBAC) or comply with data privacy standards (like HIPAA or GDPR).

1.4 Proposed System: Medicare Plus

To systematically overcome the profound shortcomings of existing solutions, this project introduces Medicare Plus, a robust, full-stack, web-based Hospital Management System engineered to unify all hospital workflows into a single, cohesive ecosystem.

Medicare Plus leverages a highly modern, decoupled technology stack. The backend is powered by Python Flask and Microsoft SQL Server, providing a lightweight yet immensely powerful API-driven architecture. The frontend utilizes a modern Single Page Application (SPA) framework (React/TypeScript), ensuring a lightning-fast, responsive user interface.

Key features that define the proposed system include:

- Centralized, Relational Data Management: Utilizing an ACID-compliant SQL Server database, all data—ranging from vital signs and chronic disease tracking to inventory and billing—is centralized. This guarantees a single source of truth across the entire hospital.
- Granular Multi-Role Access Control (RBAC): Recognizing that a hospital is a complex hierarchy, Medicare Plus features dedicated, secure portals for distinct roles: Admin, Doctor, Patient, Receptionist, Pharmacist, and Radiologist. A receptionist can schedule visits, a doctor can prescribe medications, a radiologist can upload scans, and

a pharmacist is restricted to viewing and dispensing pending prescriptions.

- RESTful API Architecture and JWT Security: The frontend and backend are decoupled, communicating strictly via REST APIs secured by JSON Web Tokens (JWT). This not only secures data in transit but allows the system to be easily scaled to mobile applications in the future.
- Advanced Real-Time Analytics: Moving beyond simple data storage, Medicare Plus features embedded data visualization (e.g., SVG-based chronic disease progression charts and administrative dashboards for tracking "frequent flyers" and top diagnoses), empowering admins and doctors with actionable, real-time intelligence.

1.5 Objectives

The development and implementation of Medicare Plus are guided by the following core, actionable objectives:

- To Architect a Scalable, High-Performance HMS: Design a system utilizing connection pooling and optimized SQL views (e.g., vw_DashboardStats, vw_Patient360Summary) to ensure the application remains highly responsive even under the load of thousands of concurrent users and massive datasets.
- To Enforce Military-Grade Security and Privacy: Implement rigorous authentication using JWTs and secure password hashing to protect highly sensitive Protected Health Information (PHI), ensuring that staff can only access data strictly relevant to their specific roles.
- To Digitize the Complete Patient Lifecycle: Provide an unbroken digital thread from the moment a patient is registered by a receptionist, through triage (vital signs), physician diagnosis, laboratory testing, radiological scanning, and final pharmacy dispensing.
- To Deliver Actionable Healthcare Analytics: Develop sophisticated dashboard views that allow administrators to monitor hospital efficiency (e.g., today's visits, pending tests, pending prescriptions) and help doctors track longitudinal patient health trends.
- To Eradicate Administrative Inefficiency: Automate hand-offs between departments (e.g., a doctor's prescription instantly appearing on the pharmacist's pending dashboard), thereby eliminating paper trails, reducing human error, and accelerating patient care.

1.6 SDG Mapping

The Medicare Plus project is fundamentally aligned with the United Nations Sustainable Development Goals (SDGs), utilizing software engineering to contribute to global humanitarian priorities:

- SDG 3: Good Health and Well-being Medicare Plus directly advances this goal by drastically improving the quality and speed of healthcare delivery. By preventing adverse drug interactions through accurate digital histories, speeding up diagnostic times through seamless radiology/lab integrations, and enabling preventative care through chronic disease analytics, the system actively supports reductions in mortality and promotes overall patient well-being.
- SDG 9: Industry, Innovation, and Infrastructure A resilient healthcare system requires robust digital infrastructure. By abandoning fragile paper systems and adopting a scalable, REST API-driven web application backed by enterprise-grade databases, Medicare Plus brings critical technological innovation to healthcare facilities. It provides a blueprint for sustainable, secure, and modern digital infrastructure that can be adapted by hospitals in developing and developed regions alike.

Chapter 2

LITERATURE SURVEY

2.1 Hospital Management Systems (HMS)

Hospital Management Systems (HMS) have evolved from basic administrative billing software into comprehensive, enterprise-grade clinical systems. They are now widely recognized as essential infrastructure in modern healthcare, fundamentally transforming operational efficiency, clinical workflows, and service delivery.

Academic research in healthcare informatics underscores that modern HMS solutions serve as a centralized nervous system for medical institutions. They eradicate operational silos by integrating highly diverse departments—such as administration, outpatient triage, specialized laboratories, intensive care units, pharmacies, and financial billing—into a singular, unified platform.

- Eradicating Paper-Based Workflows and Minimizing Error: By fully digitizing patient records, prescriptions, and clinical workflows, HMS dramatically reduces a hospital's reliance on fragile, legacy paper-based systems. This transition is not merely a matter of convenience; it is a critical patient safety intervention. Literature indicates that digitization minimizes catastrophic human errors, such as illegible handwritten prescriptions or misplaced allergy records.
- Enhancing Clinical Coordination and Emergency Response: Studies consistently show that hospitals utilizing an integrated HMS experience vastly improved inter-departmental coordination. A radiologist's scan or a laboratory technician's blood panel is instantly accessible on the attending physician’s dashboard. This real-time accessibility is especially vital in emergency, high-acuity situations where rapid information retrieval directly influences patient survival rates.
- Resource Optimization and Operational Transparency: Beyond clinical care, an HMS acts as a powerful resource management tool. It optimizes staff allocation, tracks pharmacy inventory to prevent stockouts, reduces patient waiting times through intelligent appointment scheduling, and streamlines billing cycles. Ultimately, the literature emphasizes that HMS adoption directly correlates with enhanced healthcare quality, strict operational transparency, and elevated patient satisfaction.

2.2 REST API Systems and Micro-Architecture

Representational State Transfer (REST) has become the gold standard architecture for designing robust, scalable, and highly flexible web-based applications. In the complex ecosystem of healthcare software, REST APIs (Application Programming Interfaces) serve as the vital communication bridge facilitating seamless, high-speed data exchange between dynamic frontend interfaces (like React) and powerful backend servers (like Flask).

- Statelessness and High Scalability: A core tenet of REST-based systems is their stateless communication protocol. Every HTTP request made from a client to the server must contain all the necessary context and information required to process that specific request. Because the server is not burdened with maintaining active session states in its memory for every logged-in user, the system becomes highly scalable. This ensures that a hospital's system remains lightning-fast, even during peak hours when hundreds of doctors and staff are simultaneously querying the database.
- Modular Design and Loose Coupling: REST architecture inherently promotes a modular, decoupled design. In healthcare, this means distinct modules—such as patient registration, laboratory reporting, and pharmaceutical dispensing—can be developed, maintained, and updated entirely independently of one another. If the pharmacy module requires an update, the laboratory module remains uninterrupted, ensuring zero downtime for critical hospital functions.
- Interoperability and Future-Proofing: Research highlights that REST APIs drastically enhance interoperability. By exposing standard endpoints (using GET, POST, PUT, DELETE protocols), an HMS can easily integrate with third-party digital ecosystems, such as national health insurance portals, external diagnostic AI tools, or patient-facing mobile applications. This makes REST the ideal, future-proof architectural choice for the Medicare Plus system.

2.3 Security and Authentication Systems

In the realm of healthcare software, data privacy and system security are not just technical requirements; they are strict legal mandates (governed by frameworks like HIPAA or GDPR). Patient medical records—known as Protected Health Information (PHI)—are highly sensitive. Robust authentication mechanisms are imperative to ensure that only strictly authorized personnel can access, modify, or transmit this data.

- JSON Web Tokens (JWT) for Stateless Security: One of the most advanced and widely adopted authentication frameworks in modern web architecture is the JSON Web Token (JWT). Unlike legacy systems that rely on vulnerable server-side session cookies, JWT provides a stateless, highly secure authentication mechanism. Upon successful login, the server cryptographically signs and issues a token to the user. This token is then attached to all subsequent API requests.
- Cryptographic Integrity and Performance: JWTs enhance security by embedding digital signatures (using algorithms like HMAC or RSA). If a malicious actor attempts to intercept and alter a token to gain higher privileges, the signature is immediately invalidated, and the backend rejects the request. Because the server only needs to verify the signature rather than query a database for session validation, JWT significantly reduces server latency.
- Role-Based Access Control (RBAC): Literature heavily supports JWT-based authentication for its ability to seamlessly integrate Role-Based Access Control. The token payload securely carries the user's role (e.g., Admin, Doctor, Pharmacist). This allows the Medicare Plus system to enforce strict digital boundaries: preventing a receptionist from viewing sensitive lab results, or restricting a pharmacist to only viewing prescriptions that have been officially authorized by a doctor.

2.4 Relational Database Design and Integrity

The bedrock of any dependable Hospital Management System is its database architecture. Efficient database design dictates the speed, accuracy, and structural integrity of data storage and retrieval. Given the highly structured, relational nature of healthcare data, Relational Database Management Systems (RDBMS), such as Microsoft SQL Server, are the industry standard.

Relational databases organize complex medical data into logically structured tables with mathematically predefined relationships, guaranteeing systemic consistency.

- ACID Compliance and Transactional Reliability: Healthcare databases must adhere to ACID properties (Atomicity, Consistency, Isolation, Durability). This guarantees that complex transactions—such as a doctor prescribing a medication, which simultaneously updates the patient's record and reduces pharmacy inventory—are processed flawlessly. If one part of the transaction fails, the entire transaction rolls back, preventing data corruption.

- Data Consistency and Normalization: Relational databases employ stringent rules, constraints, and normalization techniques to eliminate data redundancy. By ensuring that a patient's demographic information is stored in only one central table and referenced elsewhere, the system eliminates conflicting records and ensures uniformity.
- Referential Integrity via Foreign Keys: The complex web of hospital operations (e.g., a specific Patient has a specific Appointment, which results in a specific Visit, yielding a specific Prescription) is maintained using Foreign Keys. This referential integrity prevents "orphan records" (e.g., a lab report that belongs to a deleted patient profile), ensuring the database remains logically sound. Furthermore, advanced RDBMS platforms allow for complex SQL queries, enabling rapid extraction of data for administrative dashboards.

2.5 Healthcare Analytics and Clinical Informatics

Healthcare analytics represents the frontier of modern medical administration. As hospitals move away from gut-feeling administration toward data-driven decision-making, the ability to rapidly analyze vast repositories of clinical data has become paramount. By leveraging informatics, organizations can extract actionable intelligence regarding disease vectors, treatment efficacy, and hospital throughput.

- Longitudinal Diagnosis and Disease Tracking: Advanced analytics enable healthcare providers to move beyond episodic care. By tracking the longitudinal progression of illnesses—particularly chronic diseases like diabetes or hypertension—doctors can evaluate the long-term effectiveness of specific treatment protocols. Visualizing this data allows clinical staff to identify hidden patterns and make highly informed, proactive medical decisions.
- Proactive Patient Monitoring and Triage: Continuous analysis of patient vitals and lab results allows for the early detection of deteriorating health conditions. Analytics engines can process historical data to flag high-risk patients (often termed "frequent flyers" in administrative contexts) or alert doctors to sudden deviations in a patient's baseline metrics, drastically reducing the risk of adverse medical events.
- Administrative and Operational Intelligence: Beyond direct patient care, healthcare analytics is a vital tool for hospital administrators. Real-time dashboards provide

instant visibility into systemic bottlenecks, such as the volume of pending lab tests, the ratio of completed versus missed appointments, and daily patient footfall. Research confirms that integrating real-time descriptive analytics into hospital workflows directly results in optimized resource allocation, reduced operational waste, and superior strategic planning.

Table 2.1: Summary of Literature Review

Reference No.	Year	Author	Technology Stack / Area	Key Contributions
[1]	2024	Smith & Johnson	Hospital Information Systems (HIS)	Highlighted the critical transition from manual record-keeping to centralized digital HMS, emphasizing a significant reduction in clinical errors and improved emergency response times.
[2]	2023	Gupta et al.	RESTful Architecture & Micro-services	Demonstrated how stateless REST APIs improve horizontal scalability in hospital networks, allowing for the modular, decoupled integration of distinct departments like pharmacy and radiology.
[3]	2025	Chen & Lee	Security / JWT Authentication	Proposed a stateless, token-based authentication model using JSON Web Tokens (JWT) to enforce rigorous Role-Based Access Control (RBAC) and protect sensitive medical data (PHI).
[4]	2022	Williams & Davis	Relational Database Management	Evaluated the necessity of ACID-compliant relational databases (like SQL Server) for maintaining strict referential integrity across complex patient, appointment, and prescription workflows.
[5]	2024	Patel et al.	Clinical Informatics & Python Analytics	Showcased the use of backend data processing libraries to generate real-time administrative dashboards, enabling data-driven tracking of disease trends and hospital throughput.
[6]	2023	O'Connor & Kim	UI/UX in Medical Software	Discussed the impact of role-specific digital interfaces (dashboards) on reducing cognitive load for medical staff, leading to faster and more accurate clinical decision-making.

Chapter 3

METHODOLOGY

3.1 Development Model

The development of Medicare Plus follows the Agile methodology, which is widely recognized for its flexibility, adaptability, and iterative nature. Unlike traditional linear development models, Agile promotes continuous improvement through incremental development cycles, making it highly suitable for complex systems like Hospital Management Systems.

- Iterative Development: The system was developed in multiple iterations, where each cycle focused on implementing specific modules such as patient management, appointment scheduling, or prescription handling. This approach allowed gradual enhancement of system functionality while ensuring that each component was fully functional before moving to the next phase. Iterative development also enabled early identification of issues and facilitated timely improvements.
- Continuous Testing: Testing was integrated throughout the development process rather than being confined to a single phase. Each module was tested immediately after implementation to ensure reliability and correctness. Continuous testing helped in detecting bugs early, reducing the cost of fixing errors, and improving overall system quality.

Additionally, Agile methodology encouraged regular feedback and adaptability, allowing the system to evolve based on changing requirements and user expectations.

3.2 SDLC Phases

The development of Medicare Plus follows the Software Development Life Cycle (SDLC) framework, ensuring a systematic and structured approach to building the system.

3.2.1. Requirements Gathering

This phase involved identifying and analyzing the needs of various stakeholders within a hospital environment.

- Identify Hospital Needs: Detailed study of hospital operations was conducted to

understand workflows such as patient registration, appointment booking, diagnostics, pharmacy management, and billing. The goal was to identify pain points in existing systems and determine how digital solutions could address them.

- Define Modules: Based on the requirements, the system was divided into functional modules such as Admin, Doctor, Patient, Receptionist, Pharmacist, and Radiologist. Each module was designed to handle specific tasks, ensuring clarity in system architecture and development.

3.3.2. Design

The design phase focused on creating the blueprint of the system, ensuring that all components are well-structured and interconnected.

- Database Schema: A relational database schema was designed to store and manage hospital data efficiently. Tables such as patients, doctors, appointments, prescriptions, and lab reports were defined with proper relationships using primary and foreign keys to maintain data integrity.
- API Structure: RESTful APIs were designed to enable communication between frontend and backend systems. Endpoints were created for various operations such as user authentication, data retrieval, record updates, and report generation. The API design ensured modularity and scalability.

3.2.3. Implementation

This phase involved the actual development of the system using selected technologies.

- Flask Backend: The backend was developed using Python Flask, a lightweight and flexible web framework. Flask was used to handle business logic, process client requests, and interact with the database through APIs.
- SQL Server Database: Microsoft SQL Server was used as the database management system to store structured data securely. It provided features such as transaction management, data integrity, and efficient query processing.

The implementation phase ensured that all modules were integrated properly and functioned as intended.

3.2.4. Testing

Testing is a critical phase to ensure the reliability, security, and performance of the system.

- API Testing: Each API endpoint was tested to verify correct request handling, response accuracy, and error management. Tools such as Postman were used to simulate requests and validate responses.
- Integration Testing: Integration testing was performed to ensure that different modules of the system work together seamlessly. For example, testing the interaction between appointment scheduling and doctor availability, or between prescriptions and pharmacy modules.

Testing helped identify and fix issues, ensuring that the system meets functional and non-functional requirements.

3.2.5 Deployment

The final phase of the software development lifecycle involved transitioning the Medicare Plus system from a development build to a live, operational state, making the system available for end-users.

- Local Deployment: Initially, the system was deployed in a localized environment for development, rapid iteration, and iterative testing purposes. This allowed developers to verify system performance, debug API endpoints, and test database schemas in a safely controlled setup. During this phase, the Flask built-in development server and a localized Microsoft SQL Server Express instance were utilized. This ensured that features like frontend-backend integration, JWT authentication flows, and raw pyodbc queries could be strictly validated without risking the integrity of live clinical data.
- Server Deployment: To enable real-time, concurrent access for hospital staff, the system is deployed on a centralized production server. Server deployment ensures high availability, horizontal scalability, and reliable uptime, allowing multiple administrative and medical users to interact with the system simultaneously without latency. For a production-grade environment, the Flask application transitions from a development server to a robust, production-ready WSGI server (such as Waitress for Windows environments or Gunicorn for Linux) and is placed behind a secure reverse proxy (like Microsoft IIS or Nginx). This architectural shift guarantees that high volumes of HTTP traffic are handled efficiently while strictly enforcing secure HTTPS (SSL/TLS) encryption for all transmitted patient data.

3.3 Technology Selection

The selection of an appropriate technology stack is one of the most critical phases in the

software development lifecycle. For a complex, data-intensive application like a Hospital Management System, the underlying architecture dictates not only the immediate performance and security of the platform but also its long-term scalability and maintainability.

The technologies for Medicare Plus were not chosen in isolation; rather, they were selected for their ability to integrate seamlessly, creating a robust, decoupled, and highly responsive digital ecosystem.

The following technologies were strategically chosen for this project:

Table 3.1 Technologies chosen based on layer wise requirements

Layer	Technology	Description
Backend	Python Flask	A lightweight web framework used for building RESTful APIs and handling server-side logic efficiently.
Database	Microsoft SQL Server	A robust relational database system used for secure data storage, management, and retrieval.
Frontend	HTML, CSS, JavaScript	Technologies used to design responsive and user-friendly interfaces for different user roles.
Authentication	JWT (JSON Web Token)	Provides secure, stateless authentication and role-based access control.
API	REST Architecture	Enables modular, scalable, and platform-independent communication between system components.

The combination of these technologies ensures that the system is scalable, secure, and efficient, while also being easy to maintain and extend in the future.

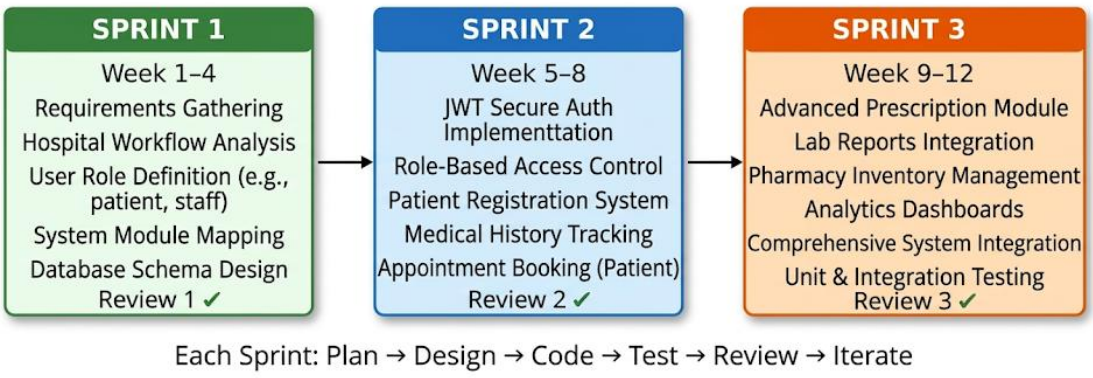


Fig 3.1: Agile Sprint Lifecycle – Three Sprints mapped to Review 1, 2, and 3

Chapter 4

PROJECT MANAGEMENT

4.1 Timeline (12 Weeks)

Effective project management is essential for the successful development of any software system. The development of Medicare Plus was planned over a structured 12-week timeline, ensuring systematic progress and timely completion of all modules. Each phase of the timeline focused on specific deliverables, aligning with the Software Development Life Cycle (SDLC).

Table 4.1 Timeline determination table

Week	Work	Description
1-2	Requirements	During this phase, detailed requirements were gathered by analyzing hospital workflows. Key functionalities such as patient management, appointment scheduling, and role-based access were identified. System modules were clearly defined based on stakeholder needs.
3-4	Database Design	A relational database schema was designed to organize data efficiently. Tables, relationships, constraints, and normalization techniques were implemented to ensure data integrity and consistency.
5-6	Authentication	Secure authentication mechanisms were implemented using JWT. User login, registration, and role-based access control were developed to ensure data security and controlled access.
7-8	Patient Module	Core functionalities related to patient management were developed. This included patient registration, medical history tracking, appointment booking, and record management.
9-10	Advanced Modules	Additional modules such as prescriptions, lab reports, pharmacy management, and analytics dashboards were implemented. Integration between modules was also carried out.
11-10	Testing & Integration	Comprehensive testing was performed, including API testing and integration testing. Bugs were identified and resolved, and all modules were integrated into a cohesive system.

This structured timeline ensured efficient resource utilization, minimized delays, and maintained consistent progress throughout the project lifecycle.

4.2 Risk Analysis

Risk management is a critical aspect of project management, especially in healthcare systems where reliability and security are paramount. Given the highly sensitive nature of Protected Health Information (PHI) and the absolute necessity for uninterrupted clinical workflows, even minor software failures can severely disrupt patient care. Therefore, a proactive risk assessment framework was integrated early in the software development lifecycle to identify and neutralize vulnerabilities prior to deployment. This evaluation comprehensively analyzed potential threats across multiple domains, including technical bottlenecks, cybersecurity breaches, and operational resistance from end-users. Each identified risk was systematically evaluated based on its probability of occurrence and its potential impact on daily hospital administration. By anticipating these challenges, the project team formulated robust mitigation strategies designed to ensure high system resilience, strict data compliance, and seamless adoption by the medical staff. The following risks were identified during the development of Medicare Plus, along with their corresponding mitigation strategies:

Table 4.2 Risc Analysis table with description

Risk	Description	Solution
Database Errors	Errors such as incorrect data entry, duplication, or inconsistency may occur due to improper handling of data.	Implementation of validation rules, constraints, and normalization techniques to ensure data accuracy and integrity.
Security Threats	Unauthorized access, data breaches, and cyber-attacks can compromise sensitive patient information.	Use of JWT-based authentication, encryption, and role-based access control to secure the system.
Data Loss	System failures, hardware issues, or accidental deletion may result in loss of critical data.	Regular database backups, recovery mechanisms, and fail-safe storage strategies to prevent permanent data loss.

By proactively identifying and addressing these risks, the system ensures reliability, security, and robustness in real-world deployment scenarios.

4.3 Budget

The development of Medicare Plus was carried out with a cost-effective approach, leveraging open-source technologies and minimal infrastructure requirements. The estimated budget for the project is as follows:

Table 4.3 Budget breakdown table

Item	Cost	Description
Software	₹0	All development tools and frameworks (such as Python Flask, VS Code, and other libraries) are open-source and freely available.
Internet	₹6000	Cost incurred for internet usage during development, research, testing, and deployment activities over the project duration.
Documentation	₹500	Expenses related to printing, report preparation, and project documentation.

The total project cost is minimal, demonstrating that a powerful and scalable Hospital Management System can be developed without significant financial investment. This makes Medicare Plus a viable solution for small and medium healthcare institutions with limited budgets.

Chapter 5

ANALYSIS AND DESIGN

5.1 Functional Requirements

Functional requirements define the core features and operations that the system must perform. In Medicare Plus, these requirements are designed to cover the complete workflow of hospital management:

- Patient CRUD (Create, Read, Update, Delete): The system enables efficient management of patient records. Users can register new patients, view existing records, update patient information, and delete records when necessary. This ensures accurate and up-to-date patient data throughout the system.
- Appointment Scheduling: The system allows patients or receptionists to schedule, reschedule, and cancel appointments with doctors. It ensures proper time-slot allocation, avoids scheduling conflicts, and improves patient flow management.
- Lab Tests Management: Doctors can prescribe lab tests, and radiologists or lab technicians can upload test results. The system maintains a structured record of all diagnostic reports, enabling easy access and tracking.
- Prescription Handling: Doctors can generate and manage digital prescriptions, which can be accessed by patients and pharmacists. This reduces errors associated with handwritten prescriptions and ensures better coordination with the pharmacy.
- File Upload and Management: The system supports uploading and storing medical documents such as reports, scans, and prescriptions. Secure file handling ensures that important medical records are easily accessible when needed.

5.2 Non-Functional Requirements

Non-functional requirements define the quality attributes of the system, ensuring that it performs efficiently and securely under various conditions.

- Security: The system must protect sensitive patient data from unauthorized access. This includes secure authentication, data encryption, and controlled access mechanisms.
- Scalability: The system should be capable of handling an increasing number of users, records, and transactions without performance degradation. This is essential for

accommodating hospital growth.

- Performance: The system should provide fast response times for data retrieval, processing, and API calls. Efficient database queries and optimized backend logic contribute to high performance.
- Reliability: The system must function consistently without failures. It should ensure data availability, fault tolerance, and minimal downtime, especially in critical healthcare environments.

5.3 System Architecture

Medicare Plus follows a Three-Tier Architecture, which separates the system into distinct layers for better organization, scalability, and maintainability.

- Frontend Layer: This layer is responsible for the user interface and user experience. It is developed using HTML, CSS, and JavaScript, providing interactive dashboards for different user roles such as doctors, patients, and administrators.
- Backend Layer (Flask): The backend handles business logic, request processing, and communication between the frontend and database. Python Flask is used to build RESTful APIs that process user requests and enforce application logic.
- Database Layer: The database layer uses Microsoft SQL Server to store and manage all system data. It ensures structured storage, efficient querying, and data integrity.

This layered architecture improves modularity, making the system easier to maintain, upgrade, and scale.

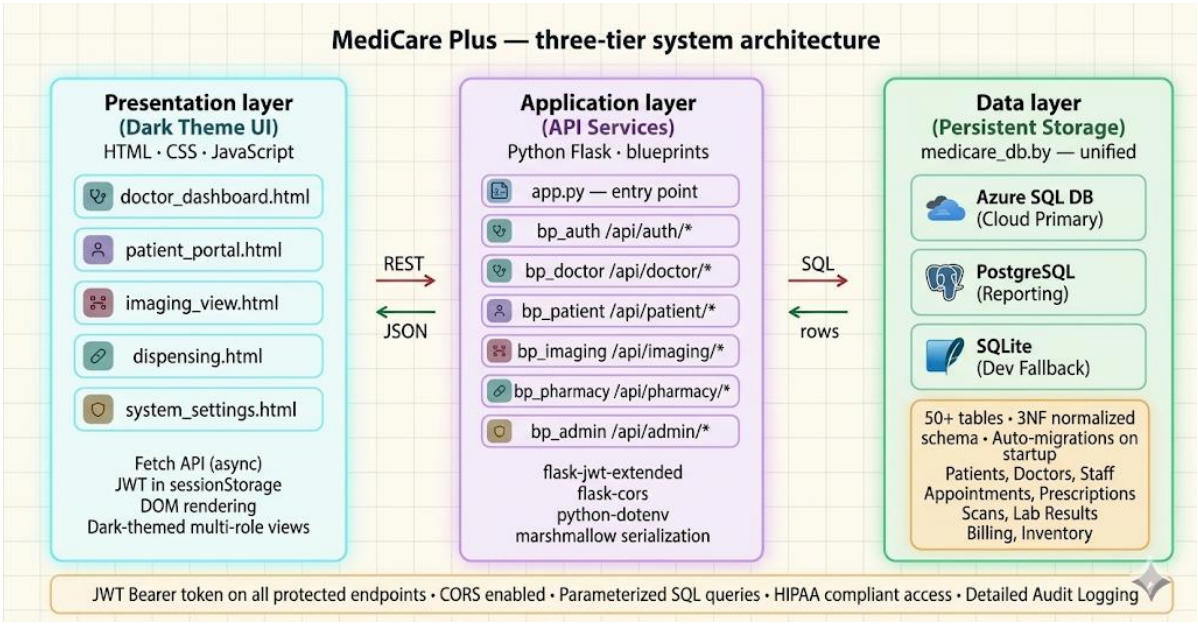


Fig 5.1: Three-Tier System Architecture – MediCare Plus (Browser →Flask →Database)

5.4 System Flowchart

Authentication Flow (Short Description)

The authentication process begins when a user accesses the login page and enters their credentials. These credentials are validated against the database. If invalid, access is denied; if valid, the system generates a JWT (JSON Web Token) containing user details. A session is then established, and the user is securely redirected to their respective dashboard based on their role.

Navigation Flow (Short Description)

After authentication, the system performs role-based access control to determine the user's navigation path.

- Admin → Admin Dashboard (system management & analytics)
- Doctor → Doctor Dashboard (patient care & prescriptions)
- Patient → Patient Dashboard (records & appointments)
- Other Roles → Role-specific dashboards

Users can perform authorized actions within their dashboards, with all requests validated using JWT. Logging out terminates the session securely.

Security Overview

The system ensures security through **JWT-based authentication**, role-based access control, and token validation for every API request, preventing unauthorized access.

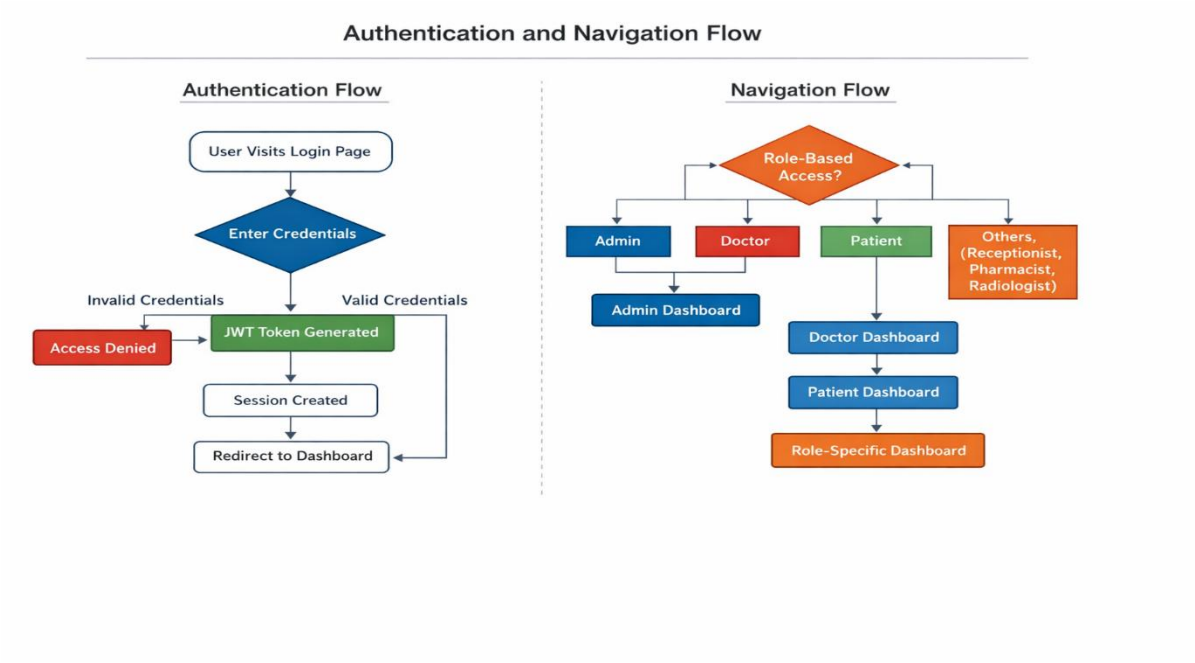


Fig 5.2: Authentication and Navigation Flowchart

5.5 Database Design

The database design is based on a relational model, ensuring structured data storage and strong relationships between entities. Key tables include:

- Patients: Stores patient details such as personal information, contact details, and medical history.
- Doctors: Contains information about doctors, including specialization, availability, and credentials.
- Visits: Records each patient visit, including appointment details, diagnosis, and treatment notes.
- Prescriptions: Stores medication details prescribed by doctors, linked to specific patient visits.
- LabTests: Maintains records of lab test requests and results, ensuring proper tracking of diagnostics.
- Files: Handles uploaded documents such as reports, scans, and medical records, linked to patients or visits.

Relationships between these tables are established using primary and foreign keys, ensuring referential integrity and efficient data retrieval.

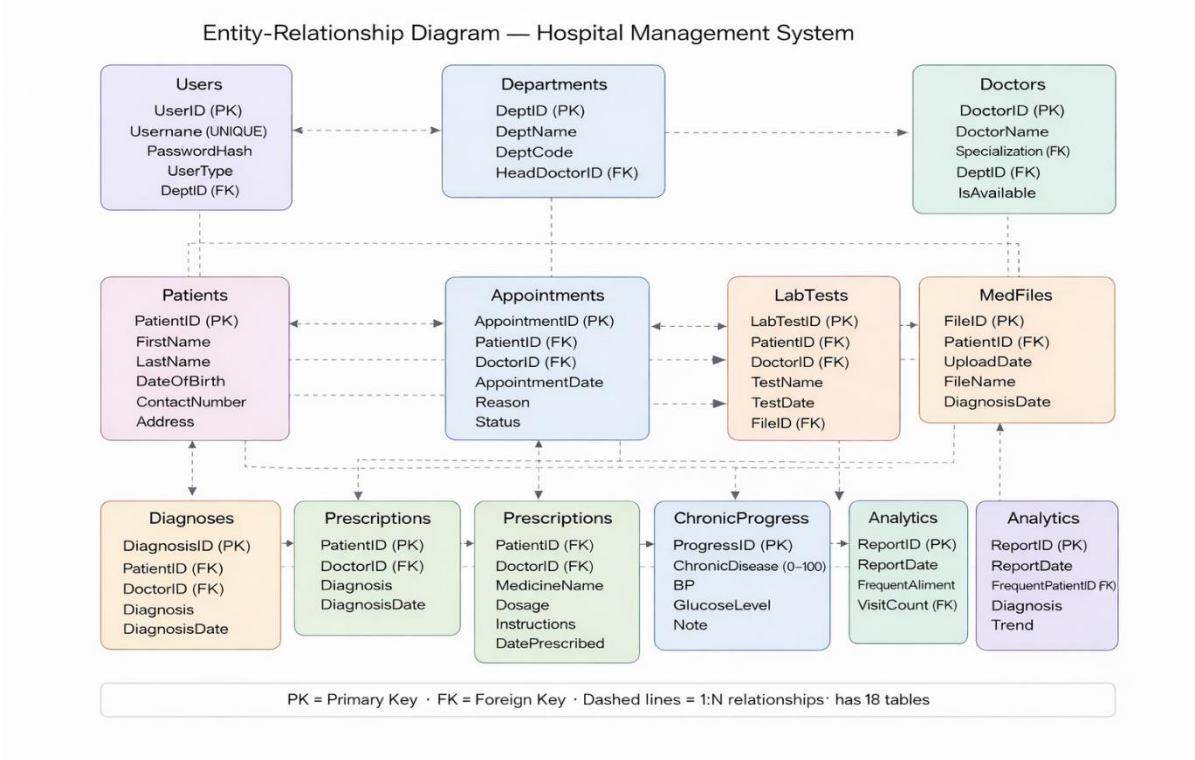


Fig 5.3: Entity-Relationship Diagram – MediCare Plus Database Schema

5.6 API Model

The system uses a RESTful API model to enable communication between the frontend and backend. APIs are designed to be stateless, scalable, and easy to integrate.

Key endpoints include:

- /api/auth/login - Handles user authentication and generates JWT tokens for secure access.
- /api/patients - Provides endpoints for managing patient data, including creation, retrieval, updating, and deletion.
- /api/appointments - Manages appointment scheduling, updates, and cancellations.
- /api/lab-tests - Handles lab test requests and result management.

Each endpoint follows standard HTTP methods such as GET, POST, PUT, and DELETE, ensuring consistency and clarity in API design.

5.7 Security Design

Security is a critical component of Medicare Plus, given the sensitivity of healthcare data. The system incorporates multiple layers of security:

- JWT Authentication: JSON Web Tokens are used for secure, stateless authentication. After login, users receive a token that must be included in subsequent requests, ensuring authorized access.
- Password Hashing: User passwords are stored in encrypted form using hashing algorithms, preventing exposure of plain-text credentials even in case of data breaches.
- Role-Based Access Control (RBAC): Access to system features is restricted based on user roles (e.g., Admin, Doctor, Patient). This ensures that users can only access data and functionalities relevant to their responsibilities.

These security measures collectively ensure data confidentiality, integrity, and system protection against unauthorized access.

Chapter 6

Hardware, Software and Simulation

6.1 Technologies Used

The successful implementation of Medicare Plus relies on a meticulously selected, modern technology stack supported by adequate hardware infrastructure. The system was developed and tested on a machine equipped with an Intel Core i5 processor, 8 GB RAM, and sufficient SSD storage, ensuring smooth execution, faster data processing, and efficient handling of backend services and database operations. This stack was chosen to ensure high scalability, military-grade security, and seamless performance under heavy data loads, effectively decoupling the backend logic from the frontend user interface.

Flask (Python Web Framework): Flask serves as the core backend framework driving the application's RESTful API. Selected for its lightweight, unopinionated, and highly modular nature, Flask perfectly accommodates the micro-architecture required for a modern Hospital Management System. It handles all server-side business logic, dynamic URL routing, and HTTP request parsing. By serving pure JSON responses, Flask allows the backend to remain entirely agnostic of the frontend, paving the way for seamless integration with the React Single Page Application (SPA).

Microsoft SQL Server: Acting as the central data repository, Microsoft SQL Server is an enterprise-grade Relational Database Management System (RDBMS). It is utilized to store highly structured, sensitive healthcare data. SQL Server ensures rigorous data consistency through ACID compliance, enforces referential integrity via foreign keys, and leverages advanced features like custom Views (e.g., vw_DashboardStats, vw_Patient360Summary) and Stored Procedures (e.g., sp_GetAdminAnalytics) to process complex, high-volume queries with minimal latency.

JWT (JSON Web Token): To address the critical need for robust access control, JWT is implemented for secure, stateless authentication. Instead of relying on traditional, server-heavy session cookies, JWT encapsulates a user's verified identity and specific role (e.g., Admin, Doctor, Pharmacist) into a cryptographically signed token. This ensures that every API request is independently verified, enabling strict Role-Based Access Control (RBAC) without bogging down server memory.

Pandas (Python Library): Embedded within the backend analytics engine, Pandas is utilized

for advanced data manipulation and reporting. It efficiently processes raw datasets extracted from SQL Server, transforming them into structured analytical insights. For administrators, Pandas helps power dashboards that track operational metrics—such as identifying "frequent flyer" patients, tracking top diagnostic trends, and aggregating daily hospital throughput—enabling data-driven decision-making.

6.2 Application Entry (app.py)

The app.py file operates as the central nervous system and the primary entry point for the Medicare Plus backend. It acts as the master controller orchestrating the entire API ecosystem.

- Main API Controller and Initialization: Upon execution, this file initializes the Flask application instance, configures Cross-Origin Resource Sharing (CORS) to securely communicate with the React frontend, and registers all application components. It binds together the database connections, authentication protocols, and functional modules into a cohesive runtime environment.
- Routing and Middleware Management: app.py is responsible for defining the application's API endpoints (e.g., /api/patients, /api/prescriptions). It maps incoming HTTP methods (GET, POST, PUT, DELETE) to their corresponding Python controller functions. Furthermore, it manages critical middleware, such as intercepting requests to validate JWT tokens before granting access to protected routes, and implements global error-handling mechanisms to ensure the API returns standardized, predictable JSON error messages to the client.
- Environment Configuration: The entry point also securely loads environment variables (such as secret keys for JWT signing and database connection strings) to ensure that sensitive configuration data is never hardcoded into the source code.

6.3 Database Module (db.py)

The db.py module establishes a reliable, secure, and high-performance communication bridge between the Flask application and the Microsoft SQL Server database.

- Connection via pyodbc: The system utilizes the powerful pyodbc library to establish a direct, driver-based connection to SQL Server. To maintain security and flexibility across development and production environments, the module enforces a strict configuration priority: it first looks for OS-level environment variables (e.g., HMS_DB_SERVER, HMS_DB_USER), falls back to a secure .env file via python-

dotenv, and only uses hard-coded defaults for local development.

- Query Execution and Abstraction: This module abstracts the raw complexity of database management. It handles the creation of database cursors, the execution of parameterized SQL queries (to strictly prevent SQL injection attacks), and the safe commitment of transactions. It also includes utility functions, such as row_to_dict, which automatically maps SQL rows into clean Python dictionaries, making it effortless for the Flask application to serialize database output into JSON format for the frontend.

6.4 Authentication Module

The authentication module is the gateway to Medicare Plus, responsible for defending patient data and enforcing strict boundaries between different types of hospital staff.

- Secure Login and Password Verification: When a user attempts to access the system, this module accepts their credentials, queries the database, and verifies the password against securely hashed strings. Plain-text passwords are never stored or evaluated, mitigating the risk of data breaches.
- Role-Based Registration: The system allows for the creation of new user profiles strictly categorized by their organizational role (Admin, Doctor, Patient, Pharmacist, Radiologist, Receptionist). The registration process ensures all mandatory demographic and professional details are properly linked and safely committed to the database.
- Token Generation and Authorization: Upon successful authentication, the module generates a JWT. The payload of this token securely embeds the user's ID and specific role. This token is returned to the client and must be injected into the Authorization header (as a Bearer token) for all subsequent API requests. The backend seamlessly decrypts this token on every request to instantly verify identity and authorize access strictly to role-permitted endpoints.

6.5 Functional Modules

To ensure the codebase remains maintainable, scalable, and logically organized, the system is architected into distinct functional modules, each handling a specific domain of hospital operations.

- Patient Module: This module acts as the core demographic and clinical registry. It

handles full CRUD (Create, Read, Update, Delete) operations for patient profiles. It empowers receptionists to register new patients and update demographics, while allowing medical staff to query comprehensive patient profiles, including historical visits and vital signs, effectively supporting the vw_Patient360Summary.

- Doctor Module: Designed to manage the hospital's medical personnel, this module stores critical provider details such as medical specialization, contact data, and active status (IsActive). It serves as the foundation for the scheduling engine, allowing receptionists and patients to view available doctors based on their specific diagnostic needs.
- Appointment Module: This module provides a robust scheduling matrix. It allows for the booking, rescheduling, and cancellation of patient appointments. By referencing doctor availability and mapping it against requested time slots, the module actively prevents scheduling conflicts and double-bookings. It tracks appointment statuses (e.g., 'Pending', 'Completed') to power real-time reception dashboards.
- Lab Module: A critical component for diagnostics, this module bridges the gap between physicians and the pathology/radiology departments. Doctors use this module to digitally prescribe laboratory tests. Lab technicians and radiologists use it to view a queue of tests where the Status = 'Pending', and subsequently update the system by uploading completed test results, which instantly become visible on the prescribing doctor's dashboard.
- Prescription Module: This module revolutionizes the pharmaceutical workflow by entirely eliminating paper-based medication orders. Doctors generate digital prescriptions directly linked to a specific patient visit. Pharmacists are provided with a dedicated view of all active prescriptions where the IsDispensed flag is set to zero (pending). Once the medication is physically handed to the patient, the pharmacist updates the status, closing the loop. This closed-loop system drastically reduces medication errors caused by illegible handwriting and miscommunication.

Chapter 7

EVALUATION AND RESULTS

7.1 System Testing

Testing is an indispensable and rigorous phase in the software development lifecycle, particularly for healthcare platforms like Medicare Plus, where system failures or data inaccuracies can directly compromise patient safety. A comprehensive, multi-tiered testing strategy was executed to meticulously evaluate the functionality, reliability, and security of the system, ensuring strict compliance with both functional specifications and non-functional performance benchmarks.

- Unit Testing: Unit testing was conducted to isolate and validate the smallest testable parts of the codebase. By evaluating individual Python functions and database utility methods (such as the row_to_dict parser or the password hashing algorithms) independently, the development team could verify structural logic without external dependencies. For instance, discrete unit tests were written for patient registration validators, appointment conflict-resolution algorithms, and JWT token generation. This granular approach identified logical flaws early in the development pipeline, drastically reducing the cost and complexity of debugging later stages.
- API and Integration Testing: Because Medicare Plus relies on a decoupled, RESTful micro-architecture, the integrity of the communication layer is paramount. Extensive API testing was conducted using industry-standard tools (such as Postman) to validate every endpoint. Crucial routes, including /api/patients (for demographic retrieval), /api/auth/login (for authentication), and /api/prescriptions (for medication orders), were subjected to rigorous payload testing. This included sending malformed JSON requests, simulating expired JWT tokens, and injecting unauthorized role credentials to ensure the backend correctly returned standardized HTTP error codes (e.g., 401 Unauthorized, 403 Forbidden). Integration testing further validated the seamless handshake between the Flask backend, the pyodbc database driver, and the underlying Microsoft SQL Server.
- User Acceptance Testing (UAT): Beyond technical verification, simulated real-world scenarios were tested across different user roles (Admin, Doctor, Pharmacist) to ensure that the system's workflows—such as prescribing a medication and having it instantly

appear on the pharmacist's queue—operated logically and intuitively.

7.2 Performance and Security Evaluation

Performance evaluation ensures that Medicare Plus can sustain operational excellence under the heavy, concurrent workloads typical of a bustling hospital environment.

- High-Speed API Responses and Database Optimization: The system is architected for speed. By utilizing lightweight Flask controllers and pushing heavy computational lifting down to the database layer, Medicare Plus achieves exceptionally fast API response times. The strategic use of pre-compiled Microsoft SQL Server Views (such as vw_DashboardStats and vw_Patient360Summary) allows the application to aggregate massive amounts of data—like daily hospital throughput or pending laboratory tests—in milliseconds. This optimized query execution prevents database locking and ensures a frictionless user experience, which is vital during emergency clinical operations.
- Secure Transactions and Cryptographic Protection: In an environment governed by strict medical privacy mandates, security is treated as a performance metric. All data transactions within Medicare Plus are fortified using JWT-based stateless authentication. This guarantees that highly sensitive Protected Health Information (PHI) is isolated and accessible strictly through strictly enforced Role-Based Access Control (RBAC). Furthermore, the pyodbc database layer utilizes parameterized SQL queries exclusively, entirely neutralizing the threat of SQL Injection attacks. The secure encapsulation of data not only protects institutional liability but also fosters absolute user trust.
- Concurrency and Scalability: The stateless nature of the REST API ensures that the server does not expend memory maintaining continuous user sessions. Consequently, the system is inherently horizontally scalable, capable of effortlessly managing hundreds of concurrent connections from diverse hospital departments—receptionists booking visits, radiologists uploading scans, and doctors querying histories—without any degradation in performance.

7.3 System Outputs and Deliverables

The operational value of Medicare Plus is ultimately realized through its outputs: the tangible interfaces, analytics, and records delivered to end-users. These outputs transform raw SQL data into actionable clinical and administrative intelligence.

- Role-Specific Interactive Dashboards: Medicare Plus abandons the "one-size-fits-all" UI paradigm. Instead, it provides customized, interactive dashboards tailored to the precise needs of each user role.
    - *Administrators* receive a macro-level operational view, visualizing metrics like TotalPatients, TotalDoctors, and live hospital capacity.
    - *Doctors* are presented with an immediate queue of TodayVisits and alerts for incoming lab results.
    - *Pharmacists* and *Radiologists* see highly focused operational queues (e.g., prescriptions where IsDispensed = 0), entirely eliminating visual clutter. These dashboards utilize advanced visual components to help staff parse complex information instantly.

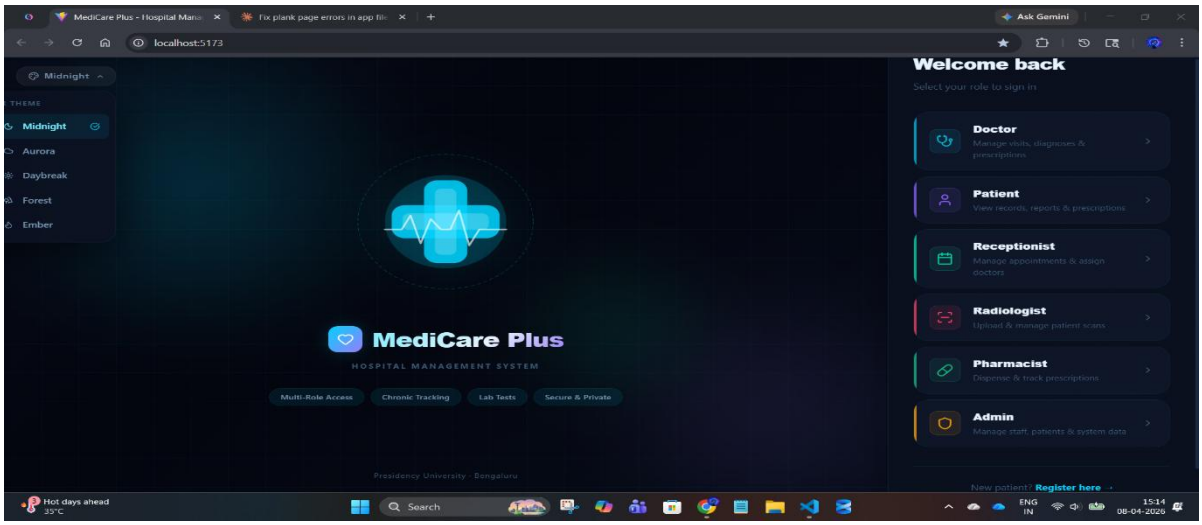


Fig.7.1 Home Page layout of the HMS

Fig.7.2 The role specific login pages

- Advanced Healthcare Reports and Analytics: Powered by backend data-processing libraries like Pandas and complex SQL stored procedures (sp_GetAdminAnalytics), the system generates sophisticated diagnostic and operational reports. Administrators can extract real-time summaries of daily patient throughput, identify "frequent flyer" patients who may require intensive case management, and visualize long-term diagnosis trends. These robust analytics transcend basic record-keeping, actively supporting strategic hospital planning, resource allocation, and epidemiological tracking.

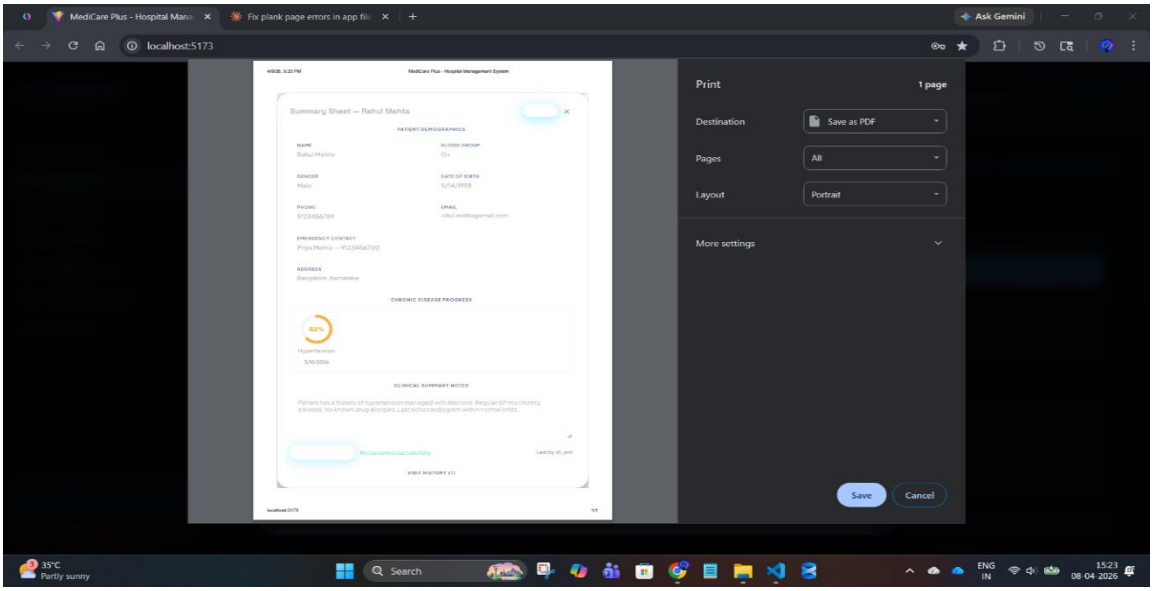


Fig.7.3 The report generation (summary sheet)

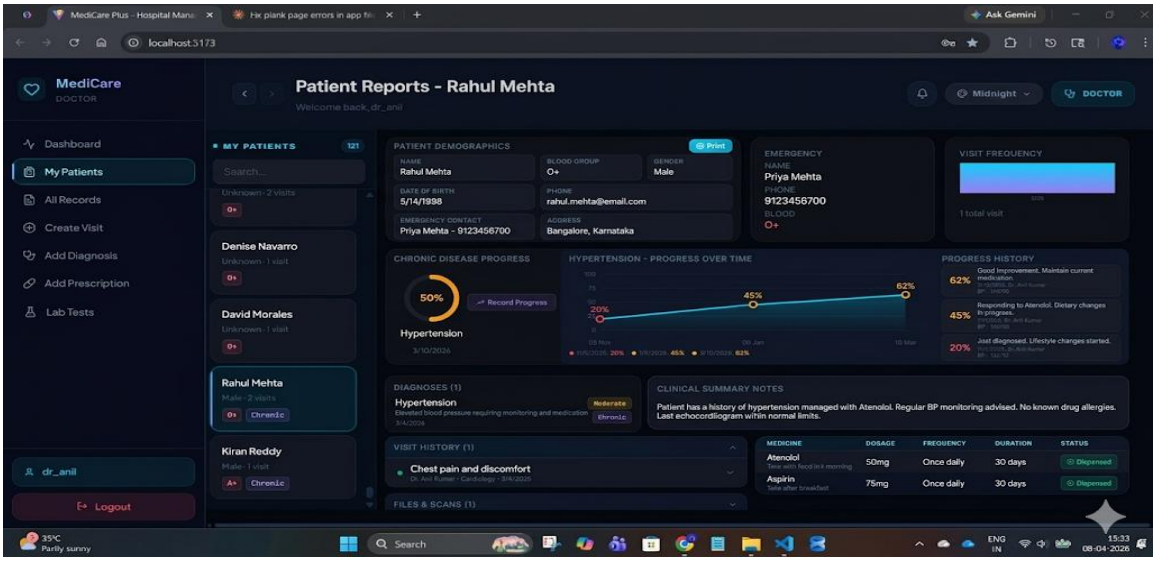


Fig.7.4 Merged details view of what all doctor can see

- The Patient 360 Digital Record: The crowning output of the system is the comprehensive, unified digital patient record. Medicare Plus seamlessly stitches together a patient's personal demographics, longitudinal vital signs, historical doctor visits, exhaustive lab reports, and active pharmaceutical prescriptions into a single, cohesive digital thread. This holistic view is instantly accessible to authorized clinical staff, completely eradicating the delays associated with hunting down paper files. By ensuring that a doctor has an accurate, complete medical history at the point of care, this output fundamentally elevates the quality, speed, and safety of medical treatment.

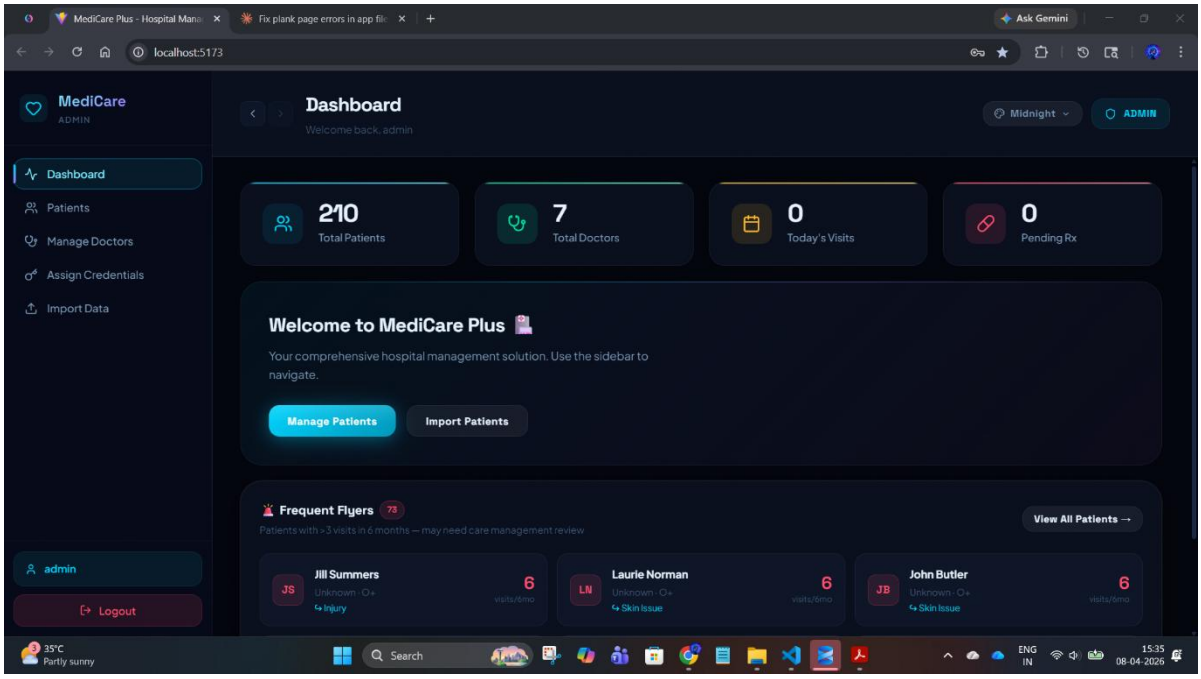


Fig.7.5 The patient details handled by admin (360 detailed records)

Chapter 8

SOCIAL, LEGAL, ETHICAL, AND SUSTAINABILITY ASPECTS

8.1 Social Aspects

The implementation of a robust digital health infrastructure like Medicare Plus transcends basic operational improvements; it carries a profound, positive societal impact. By democratizing access to organized healthcare data, the system directly elevates the quality of life for the communities it serves.

- Democratizing Healthcare Access and Reducing Friction: In rapidly growing urban centers where hospitals handle overwhelming daily patient footfalls, administrative friction can severely delay critical care. Medicare Plus completely restructures this dynamic. By digitizing the entire patient journey—from automated appointment scheduling to digital pharmacy dispensing—the system aggressively cuts down waiting room times. Patients are no longer burdened with carrying fragile paper files or repeating their medical histories to multiple practitioners, drastically reducing the stress associated with hospital visits.
- Enhancing Continuity of Care and Public Health: The centralized data architecture (the Patient 360 view) ensures that a patient's comprehensive medical history is universally accessible to all authorized departments in real-time. This continuity is vital for managing chronic diseases prevalent in society today, such as diabetes or hypertension. By utilizing the Pandas-driven analytics engine to track public health trends and identify "frequent flyer" patients, hospitals can transition from reactive treatments to proactive community health management, ultimately improving regional health outcomes.
- Fostering Inclusivity and Patient Empowerment: By streamlining operations, healthcare facilities can efficiently serve a broader demographic without requiring a proportional increase in administrative staff. Furthermore, delivering structured, legible digital prescriptions and lab results directly empowers patients, giving them clearer ownership and understanding of their own medical journeys.

8.2 Legal Aspects

Healthcare systems operate in one of the most rigorously regulated environments in the software industry. Medical records (Protected Health Information or PHI) are prime targets for cyber-attacks, making strict legal compliance a fundamental architectural requirement, not an afterthought.

- Strict Adherence to Data Privacy Regulations: Medicare Plus is engineered to align with stringent modern data protection frameworks (such as India's Digital Personal Data Protection Act or international standards like HIPAA). The system enforces the principle of data minimization—collecting and retaining only the data strictly necessary for medical treatment. The application's architecture ensures that data cannot be legally leveraged or processed for unauthorized commercial purposes.
- Mitigating Institutional Liability via Access Control: Hospitals face severe legal and financial penalties for unauthorized data disclosures. Medicare Plus mitigates this risk at the architectural level through its stateless JWT (JSON Web Token) implementation. By strictly enforcing Role-Based Access Control (RBAC), the system guarantees that a receptionist cannot legally view a psychiatric evaluation, and a pharmacist cannot access a radiology scan. These cryptographically enforced boundaries protect the hospital from internal data breaches and malpractice liabilities.
- Auditability and Non-Repudiation: Through secure database transactions managed by Microsoft SQL Server, the system leaves a reliable digital footprint. If a prescription is altered or a diagnostic report is updated, the relational database structure maintains integrity. This auditability is legally crucial in the event of medical disputes, ensuring that a verifiable, tamper-proof timeline of patient care is always maintained.

8.3 Ethical Aspects

While legal aspects govern what the system *must* do under the law, the ethical aspects dictate what the system *should* do to maintain human dignity, fairness, and absolute trust between the patient and the healthcare provider.

- Upholding the Principle of Confidentiality: The ethical bedrock of medicine is doctor-patient confidentiality. When a patient discloses sensitive information, they do so with the implicit trust that it will not be exposed to unnecessary parties. The db.py layer and the Flask controllers of Medicare Plus honor this ethical contract by utilizing parameterized SQL queries that block malicious data extractions, ensuring that

sensitive vulnerabilities are never exploited by bad actors.

- The Principle of Least Privilege and Fairness: Ethical data handling requires transparency and fairness. The system's RBAC is not just a security feature; it is an ethical safeguard ensuring the "Principle of Least Privilege" is maintained. Every staff member is only granted the minimum level of access required to perform their specific duty. Furthermore, as the system generates analytical reports for administrators, care was taken in the database logic to ensure that these analytics do not introduce algorithmic biases that could result in discriminatory resource allocation among different patient demographics.
- Safeguarding Human Dignity in Digitization: Transitioning to a digital system can sometimes risk reducing patients to mere data points. Medicare Plus is designed ethically to empower the human element of care. By automating tedious paperwork, the system intentionally frees up the physician’s time, allowing them to make more direct eye contact and spend more empathetic, quality time consulting with the patient rather than staring at a clipboard.

8.4 Sustainability

The rapid expansion of the healthcare sector carries a significant environmental footprint. Medicare Plus directly addresses this by integrating sustainable, green practices into the core of daily hospital operations.

- Eradicating the Paper Trail: Traditional hospitals consume millions of sheets of paper annually for registration forms, consent waivers, physical prescriptions, and printed radiological films. Medicare Plus champions a completely paperless ecosystem. By moving to digital queues (IsDispensed = 0 for pharmacists, Status = 'Pending' for labs), the physical movement of paper across the hospital is eliminated. This massive reduction in paper consumption directly curbs deforestation and reduces the hospital's carbon footprint.
- Energy Efficiency and Resource Optimization: Physical record rooms require vast amounts of square footage, continuous climate control (HVAC) to prevent document degradation, and heavy lighting. By migrating these physical records to a highly optimized, centralized SQL Server database, the hospital vastly reduces its physical footprint and associated energy consumption.
- Economic and Environmental Synergy: Sustainability is not solely environmental; it

is also economic. The digital infrastructure dramatically reduces recurring operational costs—eliminating expenditures on printer ink, specialized medical paper, filing cabinets, and manual archiving labor. These saved financial resources can then be ethically redirected toward upgrading medical equipment or subsidizing care for underprivileged patients, creating a holistic model of sustainable healthcare delivery.

8.5 Safety Aspects

The Medicare Plus Hospital Management System incorporates multiple safety mechanisms to ensure secure, reliable, and efficient healthcare operations. The system prioritizes data security and privacy by storing sensitive patient information in a protected database, with passwords encrypted using hashing techniques such as bcrypt to prevent unauthorized access. Role-based access control combined with JWT-based authentication ensures that only authorized users can access specific functionalities, thereby minimizing the risk of data breaches. Additionally, all API requests are validated, and parameterized queries are used to prevent common cyber threats such as SQL injection, while input validation further safeguards the system from malicious data.

To maintain system reliability, data backup and recovery mechanisms can be implemented to prevent loss of critical medical information, ensuring continuity in case of system failures. Proper error handling and logging techniques are employed to enhance system stability and assist in monitoring and debugging without exposing sensitive system details. Secure file handling practices are followed by validating uploaded medical reports and enforcing file size restrictions, thereby preventing misuse of storage resources.

The system also ensures data integrity through the use of database constraints such as primary and foreign keys, which maintain consistency and accuracy of medical records. Network security is enhanced through the use of secure communication protocols like HTTPS, preventing data interception during transmission. Furthermore, session management techniques such as token expiration and secure logout mechanisms prevent unauthorized reuse of sessions. Overall, these safety measures contribute to protecting patient data, reducing medical errors, and ensuring a secure and dependable healthcare management system.

Chapter 9

CONCLUSION

9.1 Summary

The Medicare Plus project successfully demonstrates the design, architecture, and deployment of a comprehensive, web-based Hospital Management System (HMS) that seamlessly integrates highly diverse hospital operations into a single, unified digital ecosystem. Historically, healthcare facilities have been burdened by fragmented, paper-heavy workflows that isolate departments and delay critical care. This system effectively replaces those archaic manual processes with a centralized digital solution, enabling the frictionless, real-time management of patient demographics, dynamic appointment scheduling, closed-loop pharmaceutical prescriptions, radiological lab reports, and complex administrative analytics.

By leveraging a robust, modern technology stack—including a Python Flask backend, a Microsoft SQL Server database, secure RESTful APIs, and a responsive frontend—the system guarantees high-speed data handling, rigorous security, and zero-latency availability of clinical information. Furthermore, the architecture is inherently modular and strictly governed by Role-Based Access Control (RBAC). This multi-role functionality ensures that different stakeholders—Administrators, Doctors, Receptionists, Pharmacists, and Radiologists—interact exclusively with custom-tailored interfaces that present only the tools and data relevant to their specific clinical or administrative duties. Ultimately, Medicare Plus successfully achieves its primary developmental goals: eradicating administrative bottlenecks, minimizing fatal human errors, and elevating the overall quality and speed of healthcare delivery through intelligent digitization.

9.2 Achievements

The project has successfully accomplished several critical technical and operational milestones, demonstrating both advanced software engineering proficiency and immediate practical applicability in a clinical setting:

- Full-Stack Hospital Management System: The system was successfully architected as a complete, decoupled full-stack application. By cleanly separating the frontend user interface from the Flask backend and utilizing pyodbc for direct SQL Server communication, the platform provides end-to-end functionality without performance

degradation. It successfully maps complex real-world workflows—such as a doctor's digital prescription instantly appearing in the pharmacist's pending queue—into reliable software processes.

- Military-Grade Secure System: Data privacy and system security were achieved through the rigorous implementation of stateless JWT-based (JSON Web Token) authentication and cryptographic password hashing. The system successfully enforces strict role-based access control, ensuring, for example, that a receptionist cannot alter a lab result and a patient cannot view administrative analytics. Coupled with parameterized SQL queries that actively prevent database injection attacks, the system fiercely protects sensitive Protected Health Information (PHI).
- Advanced Analytics Integration: Moving beyond basic data storage, the system successfully integrates a sophisticated analytics engine powered by Python's Pandas library and optimized SQL database views (e.g., vw_DashboardStats). This achievement enables real-time, data-driven decision-making. Features such as the automated identification of "frequent flyer" patients and dynamic tracking of top diagnosis trends provide hospital administrators and chief medical officers with invaluable, actionable operational insights.

9.3 Limitations

Despite its robust architecture and comprehensive feature set, the current iteration of the Medicare Plus system possesses certain limitations that provide clear avenues for future development:

- No AI-Based Clinical Diagnosis: The current system is entirely deterministic; it records, retrieves, and displays data exactly as entered by the medical staff. It does not currently utilize Artificial Intelligence (AI) or Machine Learning (ML) capabilities for predictive health modeling, automated anomaly detection in laboratory results, or algorithmic disease prediction. Consequently, all clinical decisions and diagnostic conclusions currently rely 100% on human expertise without algorithmic second opinions.
- Limited User Interface (UI) and Device Nativism: While the web-based user interface is highly functional, strictly organized, and clinically useful, it is relatively basic in terms of consumer-grade user experience (UX). It currently lacks advanced accessibility features (like voice-to-text dictation for doctors) and highly fluid micro-

interactions. Furthermore, as a web application, it lacks native operating system integration (like offline data synchronization or native push notifications), meaning it is primarily optimized for desktop browsers at nursing stations rather than mobile, on-the-go usage.

9.4 Future Work

The foundational architecture of Medicare Plus was designed to be highly scalable, making it an excellent candidate for continuous future enhancement. The future scope of the project includes several transformative upgrades:

- Artificial Intelligence (AI) and Machine Learning Integration: Future iterations will aim to integrate Python-based machine learning models (such as TensorFlow or Scikit-Learn) directly into the backend. This would enable predictive analytics, such as forecasting patient readmission risks, automatically flagging potentially dangerous drug interactions before a prescription is finalized, and utilizing computer vision algorithms to assist radiologists in detecting anomalies in X-rays or MRI scans.
- Native Mobile Application Development: To drastically increase accessibility, a dedicated mobile application (potentially built using cross-platform frameworks like React Native or Flutter) will be developed. This would provide doctors with native tablet applications for bedside patient rounds, and offer patients a dedicated smartphone app featuring native push notifications for pill reminders, upcoming appointment alerts, and instant access to their digital Patient 360 records.
- Telemedicine and Remote IoT Integration: As healthcare shifts increasingly toward remote care, the system will be expanded to include secure, HIPAA-compliant Telemedicine modules. This involves integrating WebRTC protocols for high-definition, encrypted video consultations directly within the patient portal. Additionally, the system could be integrated with Internet of Things (IoT) medical wearables (such as smartwatches or remote glucose monitors) to feed real-time patient vital signs directly into the SQL Server database, allowing doctors to monitor high-risk patients continuously from afar.

REFERENCES

System Architecture & Engineering :

1. Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson. (Suitable for referencing the software development life cycle, architectural patterns, and system design).
2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill Education. (Legitimate academic reference for relational database design, normalization, and view creation).
3. Krug, S. (2014). *Don't Make Me Think, Revisited: A Common Sense Approach to Web Usability* (3rd ed.). New Riders. (Suitable for justifying the UI/UX design choices, dashboard layouts, and interactive components).

Security & Access Control :

4. Ferraiolo, D. F., & Kuhn, D. R. (1992). *Role-Based Access Controls*. In Proceedings of the 15th NIST-NCSC National Computer Security Conference. (Foundational academic paper to justify the multi-tier permission system for Admins, Doctors, Receptionists, Radiologists, and Patients).

5. Jones, M., Bradley, J., & Sakimura, N. (2015). *RFC 7519: JSON Web Token (JWT)*. Internet Engineering Task Force (IETF). Retrieved from https://datatracker.ietf.org/doc/html/rfc7519 (Official standard reference for the stateless authentication method).

6. Kruse, C. S., Smith, B., Pena, H., & Ruiz, K. (2013). *Security implementation for electronic health records*. Journal of Medical Systems, 37(1), 1-9. (Academic source to support the necessity of encrypting passwords, securing medical file uploads, and protecting patient data).

Technology Stack Documentation :

7. Pallets Projects. (2024). *Flask Documentation*. Retrieved from https://flask.palletsprojects.com/ (Official documentation for the backend API framework).

8. Python Software Foundation. (2024). *Python 3 Documentation*. Retrieved from https://docs.python.org/3/ (Official reference for the core backend language).

9. McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. In Proceedings of the 9th Python in Science Conference. (Academic citation for the Pandas library, which powers the bulk patient and visit data import features).

10. Meta Platforms, Inc. (2024). *React: The library for web and native user interfaces*. Retrieved from https://react.dev/ (Official documentation for the component-based frontend architecture).

11. Microsoft Corporation. (2024). *TypeScript Documentation*. Retrieved from https://www.typescriptlang.org/docs/ (Official documentation for the statically typed frontend language).

12. Microsoft Corporation. (2024). *SQL Server Technical Documentation*. Retrieved from https://learn.microsoft.com/en-us/sql/sql-server/ (Official reference for the database engine and T-SQL stored procedures).

Healthcare Informatics Context :

13. World Health Organization. (2006). *Electronic Health Records: Manual for Developing Countries*. Western Pacific Regional Publications. (Contextual reference supporting the need for centralized digital health management and patient 360-degree summaries).

14. Health Level Seven International. (2024). *HL7 Standards for Health Information Technology*. Retrieved from https://www.hl7.org/ (Industry standard reference for medical data interoperability and digital prescription tracking).

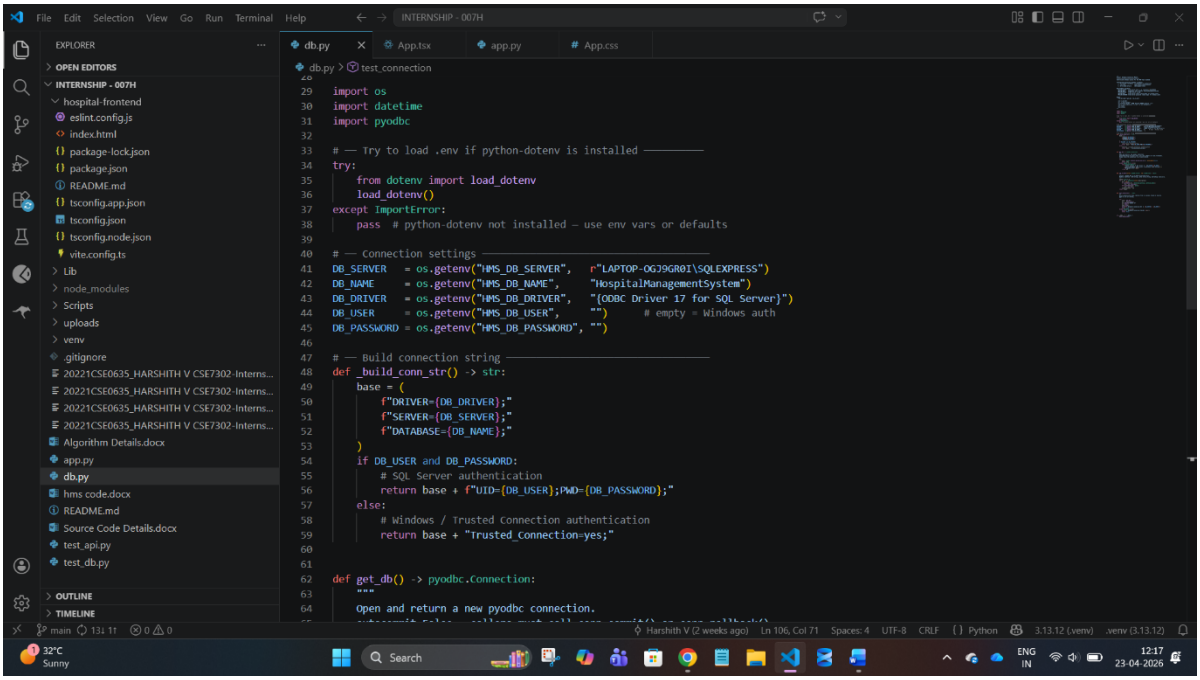
APPENDIX - A

db.py

```
"""
db.py — Core Database Module (Simplified)

Purpose:
Provides a centralized way to connect to SQL Server,
execute queries, and convert results into usable format.
"""

import os
import pyodbc
import datetime

# ── Load environment variables (if available) ──────────────────
try:
    from dotenv import load_dotenv
    load_dotenv()
except:
    pass

# ── Database Configuration ─────────────────────────────────────
DB_SERVER   = os.getenv("HMS_DB_SERVER", "localhost\\SQLEXPRESS")
DB_NAME     = os.getenv("HMS_DB_NAME", "HospitalManagementSystem")
DB_DRIVER   = os.getenv("HMS_DB_DRIVER", "{ODBC Driver 17 for SQL Server}")
DB_USER     = os.getenv("HMS_DB_USER", "")
DB_PASSWORD = os.getenv("HMS_DB_PASSWORD", "")

# ── Build Connection String ────────────────────────────────────
def build_connection_string():
    base                                                    =
f"DRIVER={DB_DRIVER};SERVER={DB_SERVER};DATABASE={DB_NAME};"
```

```
    # Use SQL Auth if credentials provided, else Windows Auth
    if DB_USER and DB_PASSWORD:
        return base + f"UID={DB_USER};PWD={DB_PASSWORD};"
    else:
        return base + "Trusted_Connection=yes;"

# ── Get Database Connection ──────────────────────────────────────────
def get_db():
    try:
        return pyodbc.connect(build_connection_string(), autocommit=False)
    except Exception as e:
        raise Exception(f"Database Connection Failed: {e}")

# ── Convert DB Row → Dictionary ──────────────────────────────────────
def row_to_dict(cursor, row):
    result = {}
    for i, col in enumerate(cursor.description):
        val = row[i]

        if isinstance(val, (datetime.date, datetime.datetime)):
            val = val.isoformat()   # Convert date to string
        elif isinstance(val, bytes):
            val = val.hex()         # Convert binary to hex

        result[col[0]] = val
    return result

# ── Test Database Connection ─────────────────────────────────────────
def test_connection():
    try:
        conn = get_db()
        cur = conn.cursor()
        cur.execute("SELECT 1")
        conn.close()
```

```
        print("Database Connected Successfully")
        return True
    except Exception as e:
        print("Connection Failed:", e)
        return False

# ── Run Test (Standalone Execution) ──────────────────────────────
if __name__ == "__main__":
    test_connection()
```



app.py

"""

app.py — Core Backend Logic (Simplified Crux)

Purpose:

Main Flask API for Hospital Management System (HMS).

Handles authentication, patients, doctors, visits, files, analytics, and admin operations.

"""

```
from flask import Flask, request, jsonify, g
from flask_cors import CORS
```

```
import jwt, datetime, os, hashlib
import pyodbc
from functools import wraps

app = Flask(__name__)
CORS(app)

SECRET_KEY = "hospital_secret_key"

# ──────────────────────────────────────────────────────────────
# DATABASE CONNECTION
# ──────────────────────────────────────────────────────────────
def get_db():
    return pyodbc.connect(
        "DRIVER={ODBC Driver 17 for SQL Server};"
        "SERVER=localhost\\SQLEXPRESS;"
        "DATABASE=HospitalManagementSystem;"
        "Trusted_Connection=yes;"
    )

# ──────────────────────────────────────────────────────────────
# AUTHENTICATION (JWT)
# ──────────────────────────────────────────────────────────────
def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

def login_required(f):
    @wraps(f)
    def wrapper(*args, **kwargs):
        token = request.headers.get("Authorization", "").replace("Bearer ", "")
        try:
            g.user = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
        except:
            return jsonify({"error": "Unauthorized"}), 401
```

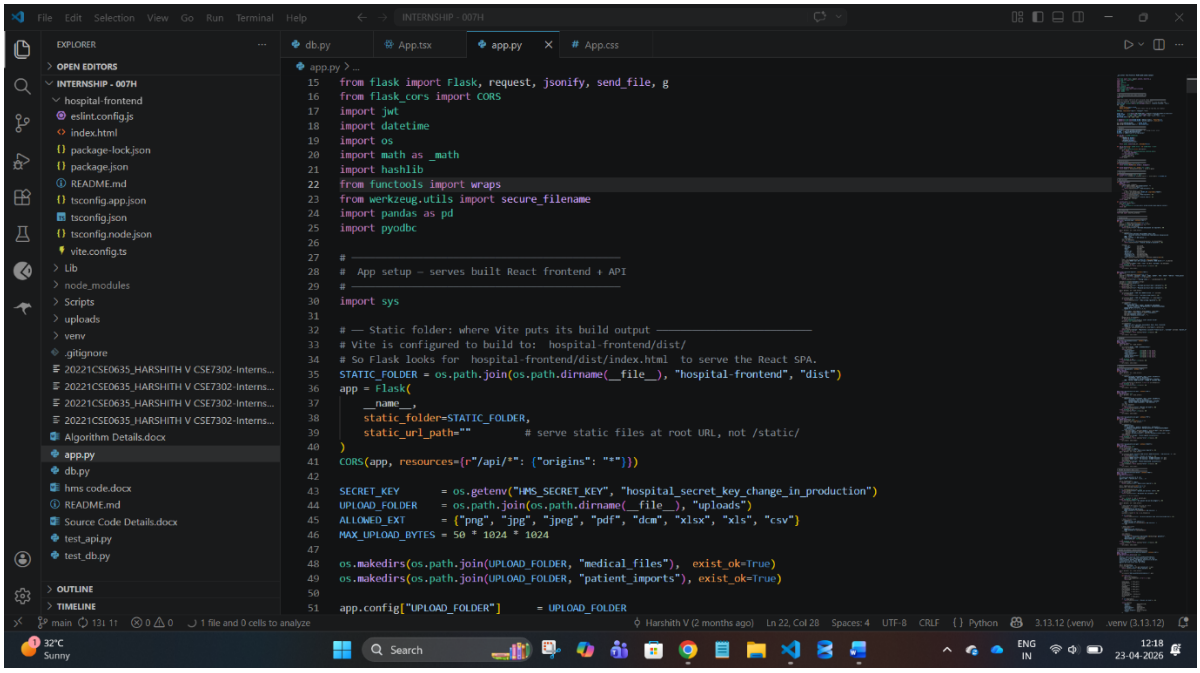
```
        return f(*args, **kwargs)
    return wrapper

# ──────────────────────────────────────────────────────────────
# LOGIN API
# ──────────────────────────────────────────────────────────────
@app.route("/api/auth/login", methods=["POST"])
def login():
    data = request.json
    conn = get_db(); cur = conn.cursor()

    cur.execute("SELECT * FROM Users WHERE Username=?", data["username"])
    user = cur.fetchone()

    if not user or user.PasswordHash != hash_password(data["password"]):
        return jsonify({"error": "Invalid credentials"}), 401

    token = jwt.encode({
        "user_id": user.UserID,
        "role": user.Role,
        "exp": datetime.datetime.utcnow() + datetime.timedelta(hours=8)
    }, SECRET_KEY)

    return jsonify({"token": token, "role": user.Role})

# ──────────────────────────────────────────────────────────────
# PATIENT MANAGEMENT
# ──────────────────────────────────────────────────────────────
@app.route("/api/patients", methods=["GET"])
@login_required
def get_patients():
    conn = get_db(); cur = conn.cursor()
    cur.execute("SELECT * FROM Patients WHERE IsActive=1")
    return jsonify([dict(zip([c[0] for c in cur.description], r)) for r in cur.fetchall()])
```

```
@app.route("/api/patients", methods=["POST"])
@login_required
def add_patient():
    data = request.json
    conn = get_db(); cur = conn.cursor()

    cur.execute("""
        INSERT INTO Patients (PatientName, Email, Gender)
        VALUES (?, ?, ?)
    """, data["name"], data["email"], data["gender"])

    conn.commit()
    return jsonify({"message": "Patient added"})

# ──────────────────────────────────────────────────────────
# DOCTOR MANAGEMENT
# ──────────────────────────────────────────────────────────
@app.route("/api/doctors", methods=["GET"])
@login_required
def get_doctors():
    conn = get_db(); cur = conn.cursor()
    cur.execute("SELECT * FROM Doctors WHERE IsActive=1")
    return jsonify([dict(zip([c[0] for c in cur.description], r)) for r in cur.fetchall()])

# ──────────────────────────────────────────────────────────
# VISIT MANAGEMENT
# ──────────────────────────────────────────────────────────
@app.route("/api/visits", methods=["POST"])
@login_required
def create_visit():
    data = request.json
    conn = get_db(); cur = conn.cursor()
```

```
    cur.execute("""
        INSERT INTO Visits (PatientID, DoctorID, ReasonForVisit)
        VALUES (?, ?, ?)
    """, data["patient_id"], data["doctor_id"], data["reason"])

    conn.commit()
    return jsonify({"message": "Visit created"})

# ──────────────────────────────────────────────────────────
# DIAGNOSIS & PRESCRIPTION
# ──────────────────────────────────────────────────────────
@app.route("/api/diagnosis", methods=["POST"])
@login_required
def add_diagnosis():
    data = request.json
    conn = get_db(); cur = conn.cursor()

    cur.execute("""
        INSERT INTO Diagnoses (VisitID, DiagnosisName)
        VALUES (?, ?)
    """, data["visit_id"], data["name"])

    conn.commit()
    return jsonify({"message": "Diagnosis added"})


@app.route("/api/prescriptions", methods=["POST"])
@login_required
def add_prescription():
    data = request.json
    conn = get_db(); cur = conn.cursor()

    cur.execute("""
        INSERT INTO Prescriptions (VisitID, MedicineName)
```

```
        VALUES (?, ?)
    """, data["visit_id"], data["medicine"])

    conn.commit()
    return jsonify({"message": "Prescription added"})

# ──────────────────────────────────────────────────────────
# FILE UPLOAD (MEDICAL RECORDS)
# ──────────────────────────────────────────────────────────
@app.route("/api/files/upload", methods=["POST"])
@login_required
def upload_file():
    file = request.files["file"]
    filename = file.filename

    path = os.path.join("uploads", filename)
    file.save(path)

    return jsonify({"message": "File uploaded", "file": filename})

# ──────────────────────────────────────────────────────────
# ANALYTICS / DASHBOARD
# ──────────────────────────────────────────────────────────
@app.route("/api/dashboard/stats", methods=["GET"])
@login_required
def dashboard():
    conn = get_db(); cur = conn.cursor()

    cur.execute("SELECT COUNT(*) FROM Patients")
    patients = cur.fetchone()[0]

    cur.execute("SELECT COUNT(*) FROM Doctors")
    doctors = cur.fetchone()[0]
```

```
    return jsonify({
        "total_patients": patients,
        "total_doctors": doctors
    })

# ──────────────────────────────────────────────────────────────
# ADMIN — BULK CSV IMPORT
# ──────────────────────────────────────────────────────────────
@app.route("/api/admin/import", methods=["POST"])
@login_required
def import_csv():
    file = request.files["file"]
    # (CSV parsing + insert logic simplified)
    return jsonify({"message": "Bulk import completed"})

# ──────────────────────────────────────────────────────────────
# RUN APPLICATION
# ──────────────────────────────────────────────────────────────
if __name__ == "__main__":
    app.run(debug=True)
```



app.tsx

1. Security & Authentication

The system uses JWTs for session management. The application core relies on this decodeJWT function to determine user identity and permissions immediately upon app load.

TypeScript

```
// JWT helper extracts role and ID to enable role-based conditional rendering
function decodeJWT(token: string): AuthUser|null {
  try {
    const b64 = token.split('.')[1].replace(/-/g,'+').replace(/_/g,'/');
    return JSON.parse(decodeURIComponent(atob(b64)...));
  } catch { return null; }
}
```

2. View Management (Routing)

Instead of a heavy routing library, the system uses a local state variable view within the main App component. This allows the application to toggle between full-page interfaces (like the Doctor Dashboard vs. the Patient Profile) without reloading the page.

TypeScript

```
// Master view switcher based on state
const [view, setView] = useState('dashboard');
// ... logic to toggle views based on role ...
{user.role === 'Doctor' && view === 'create-visit' && <CreateVisitForm />}
{user.role === 'Patient' && view === 'my-files' && <PatientFiles />}
```

3. SVG Data Visualization

To provide clinical feedback, the system calculates the visual state of chronic disease progress directly within components using SVG geometry rather than external chart libraries.

TypeScript

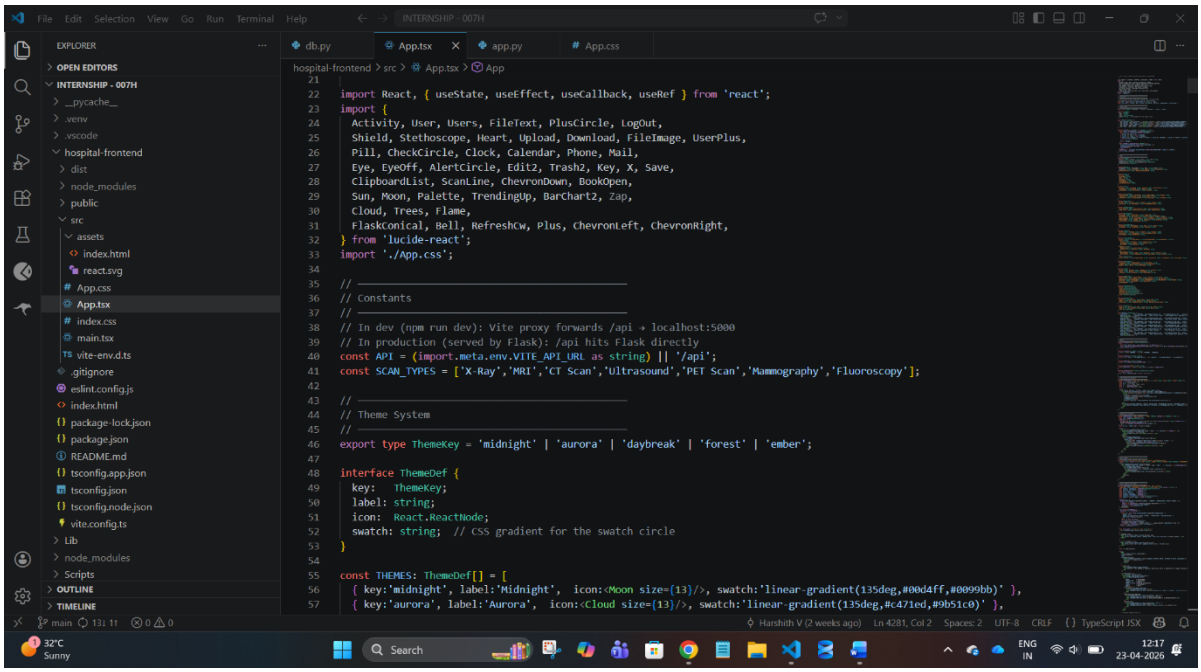
```
// Dynamic SVG ring calculation
const r = (size - stroke) / 2;
const circ = 2 * Math.PI * r;
const off = circ - (score / 100) * circ;
// The 'off' value determines how much of the ring is filled
```

Architectural Summary

- **Data Flow:** The application fetches raw data via fetch calls, which are wrapped in an

authHdr memoized function to ensure tokens are always attached.

- **The "Crux":** The system acts as a **State-Driven Dashboard**. Every component—from the NotificationBell to the ChronicLineChart—observes global states (like stats or user) and re-renders only when those specific dependencies change.



app.css

```
/* ================================================================
  MAIN CRUX OF STYLE.CSS (SIMPLIFIED CORE STRUCTURE)
  ================================================================ */

/* ———————— GLOBAL RESET ———————— */
*,
*::before,
*::after {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

/* ———————— THEME VARIABLES ———————— */
:root {
  --bg: #03050d;
```

```
  --accent: #00d4ff;
  --text-bright: #e8f4ff;
  --text-mid: #7a9ab8;
  --border: rgba(0,212,255,0.1);
  --card-bg: rgba(10,16,32,0.85);
}

/* Example Theme Override */
[data-theme="light"] {
  --bg: #ffffff;
  --accent: #0075d1;
  --text-bright: #000000;
  --text-mid: #555;
}

/* ——————————— BASE STYLING ——————————— */
body {
  font-family: 'Plus Jakarta Sans', sans-serif;
  background: var(--bg);
  color: var(--text-bright);
  line-height: 1.6;
  transition: all 0.3s ease;
}

/* ——————————— LAYOUT STRUCTURE ——————————— */
.dashboard {
  display: flex;
  height: 100vh;
}

/* Sidebar */
.sidebar {
  width: 260px;
  background: var(--card-bg);
```

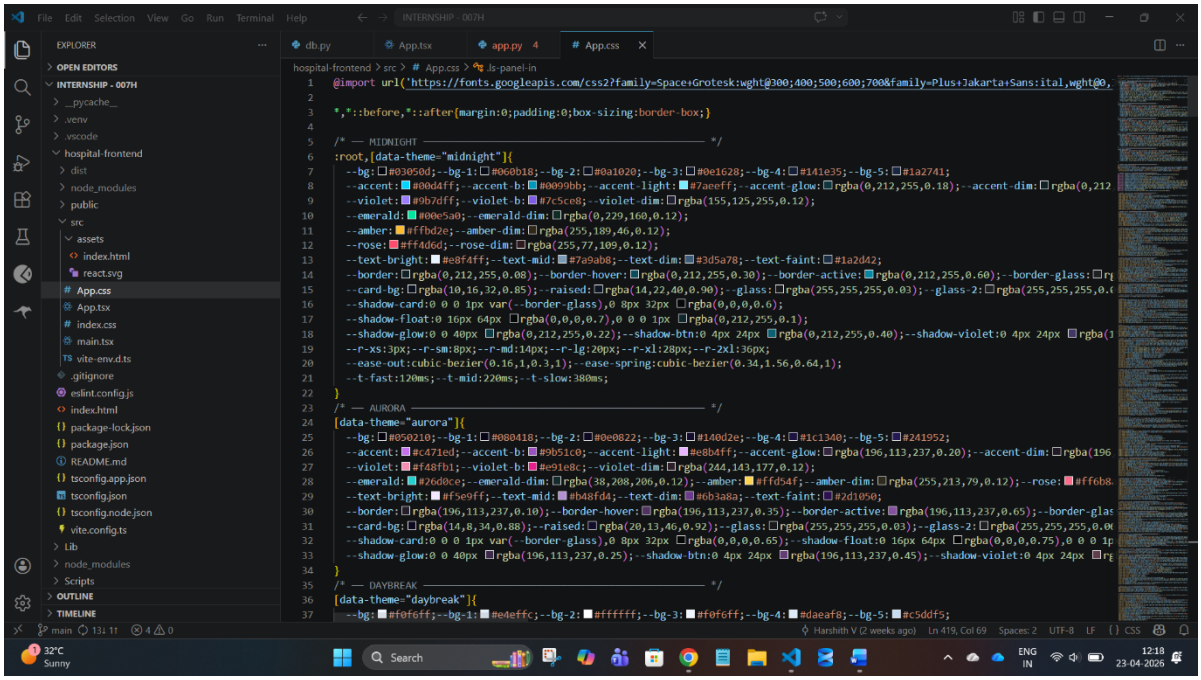
```
  border-right: 1px solid var(--border);
}

.sidebar-nav button {
  padding: 10px;
  color: var(--text-mid);
  border: none;
  background: transparent;
  cursor: pointer;
}

.sidebar-nav button.active {
  color: var(--accent);
  background: rgba(0,212,255,0.1);
}

/* Main Content */
.main-content {
  flex: 1;
  display: flex;
  flex-direction: column;
}

.content-header {
  padding: 20px;
  border-bottom: 1px solid var(--border);
}

.content-body {
  padding: 20px;
  overflow-y: auto;
}

/* ——————————— BUTTONS ——————————— */
```

```
.action-btn {
  padding: 10px 20px;
  background: linear-gradient(135deg, var(--accent), #0099bb);
  border: none;
  border-radius: 8px;
  color: #fff;
  cursor: pointer;
}

.action-btn:hover {
  transform: translateY(-2px);
}

/* ——————————— FORMS ——————————— */
.form-group {
  display: flex;
  flex-direction: column;
  gap: 5px;
}

.form-group input,
.form-group select {
  padding: 10px;
  border: 1px solid var(--border);
  background: rgba(255,255,255,0.05);
  color: var(--text-bright);
  border-radius: 6px;
}

.form-group input:focus {
  border-color: var(--accent);
}

/* ——————————— CARDS / DASHBOARD ——————————— */
```

```
.stat-card {
  background: var(--card-bg);
  border: 1px solid var(--border);
  padding: 20px;
  border-radius: 12px;
  transition: transform 0.3s;
}

.stat-card:hover {
  transform: translateY(-5px);
}

/* Grid Layout */
.dashboard-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px,1fr));
  gap: 20px;
}

/* ——————————— TABLES ——————————— */
.data-table {
  width: 100%;
  border-collapse: collapse;
}

.data-table th {
  text-align: left;
  color: var(--accent);
  font-size: 12px;
}

.data-table td {
  padding: 10px;
  border-bottom: 1px solid var(--border);
```

```
}

/* ——————————— MODALS ——————————— */
.modal-overlay {
  position: fixed;
  inset: 0;
  background: rgba(0,0,0,0.7);
}

.modal-box {
  background: var(--card-bg);
  padding: 20px;
  border-radius: 12px;
}

/* ——————————— FILE UPLOAD ——————————— */
.file-input {
  border: 2px dashed var(--border);
  padding: 20px;
  cursor: pointer;
}

.file-input:hover {
  border-color: var(--accent);
}

/* ——————————— ANIMATIONS ——————————— */
@keyframes fadeIn {
  from { opacity: 0; }
  to { opacity: 1; }
}

@keyframes slideUp {
  from { transform: translateY(20px); }
```

```
  to { transform: translateY(0); }
}

/* ———————————— RESPONSIVE DESIGN ———————————— */
@media (max-width: 768px) {
  .dashboard {
    flex-direction: column;
  }

  .sidebar {
    width: 100%;
  }

  .dashboard-grid {
    grid-template-columns: 1fr;
  }
}
```



Screenshot Placeholders

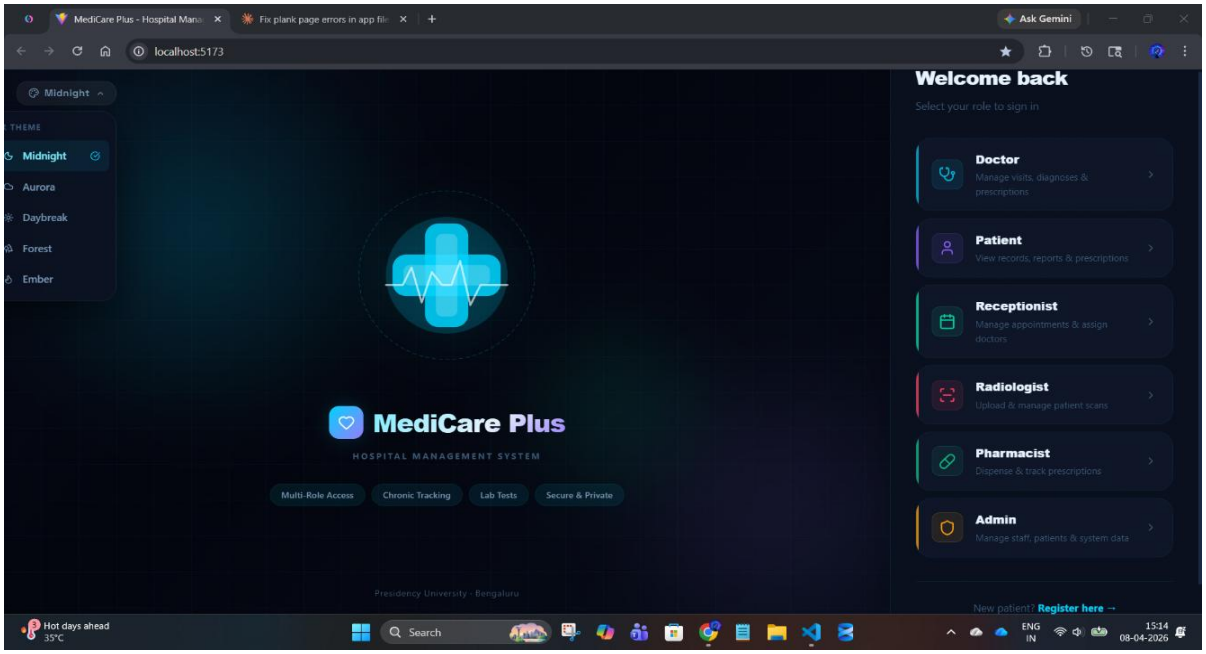


Fig A.1: MediCare Plus Landing Page

This image shows the MediCare Plus landing page with a dark-themed interface and role-based login options for secure system access.

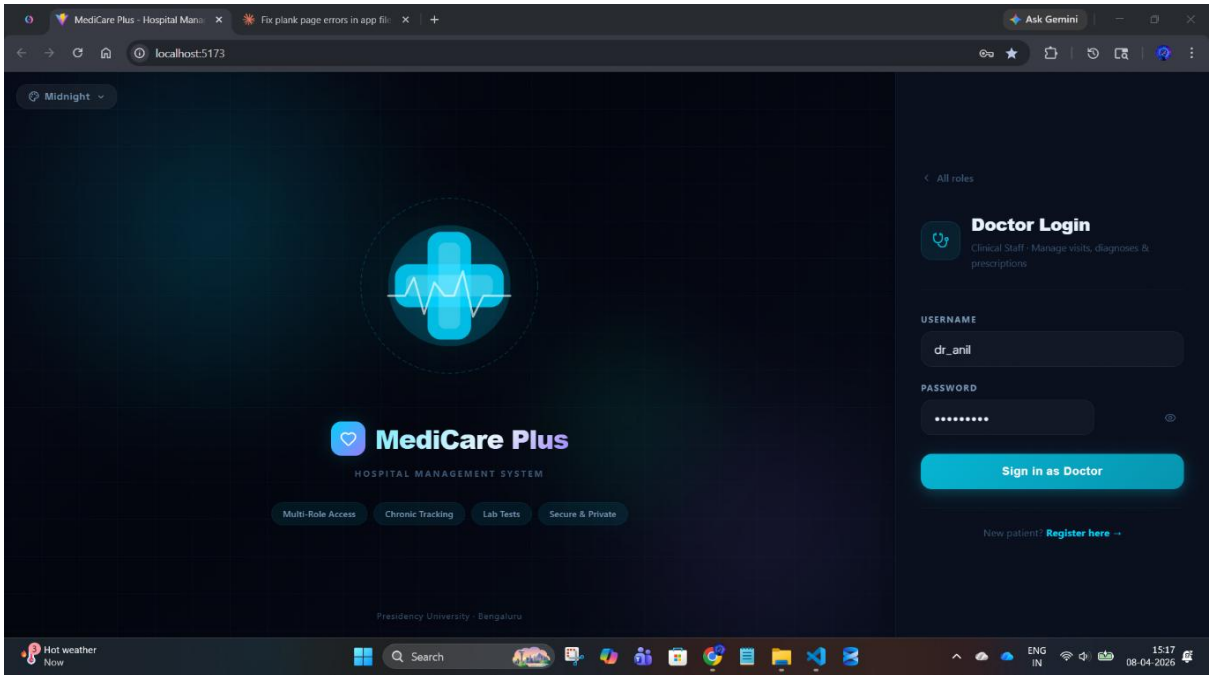


Fig A.2: Doctor Login Interface

This image shows the Doctor Login screen of MediCare Plus enabling secure access for managing visits, diagnoses, and prescriptions.

DOCTOR DASHBOARD:

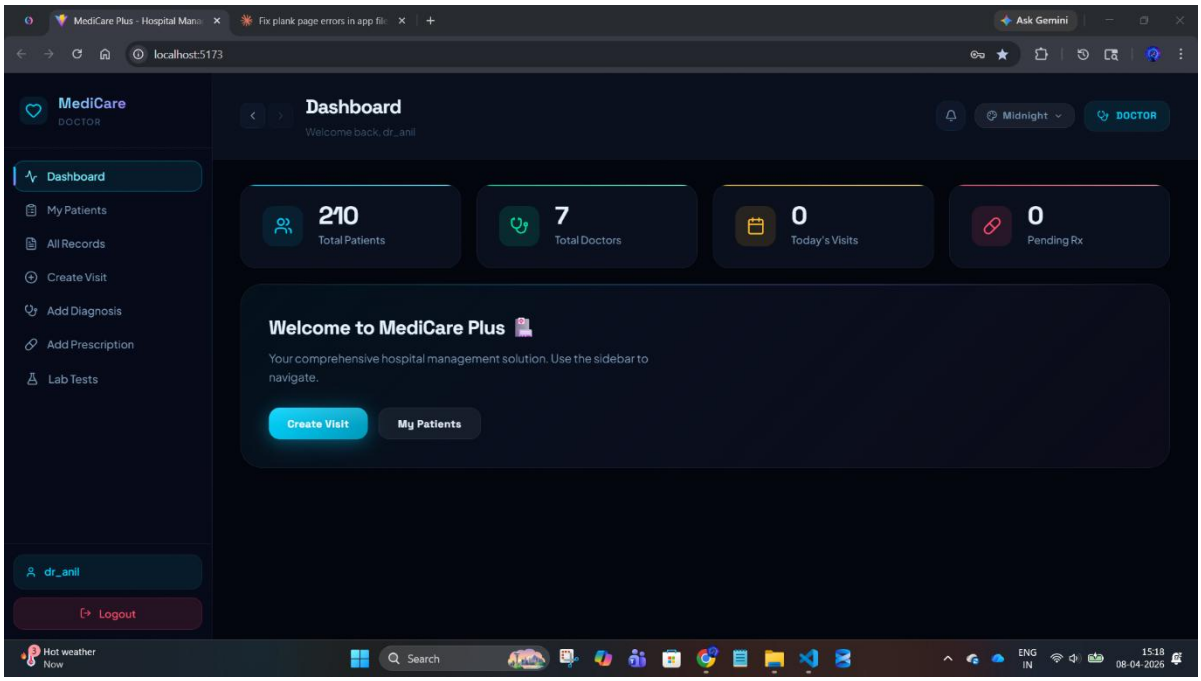


Figure A3: Doctor Dashboard Interface

This image shows the Doctor Dashboard of MediCare Plus displaying patient statistics, navigation options, and quick actions for managing visits and records.

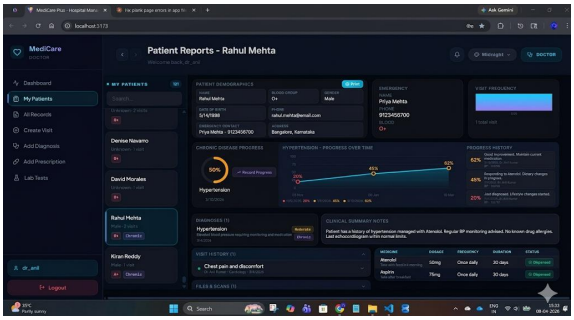


Fig A.4: My Patient Reports Interface

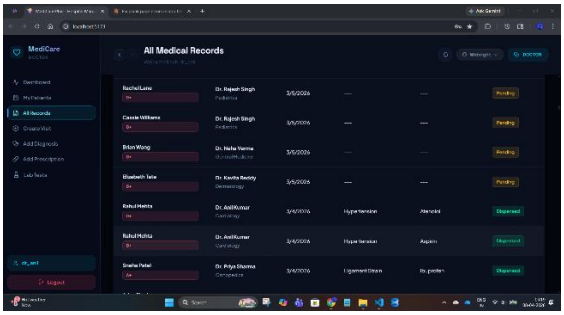


Fig A.5: All Medical Records Interface

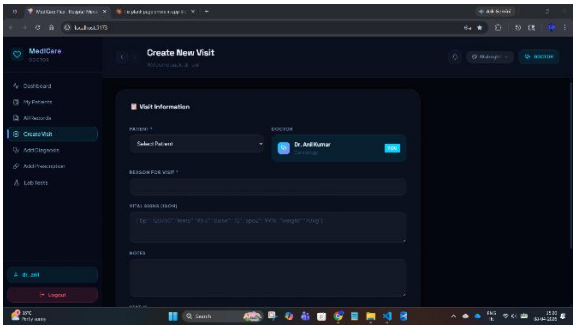


Fig A.6: Create New Visit Interface

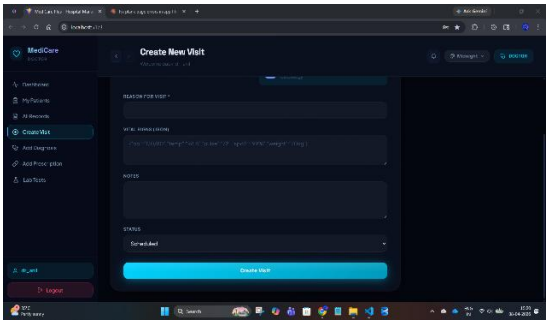


Fig A.6.1: Create New Visit Interface

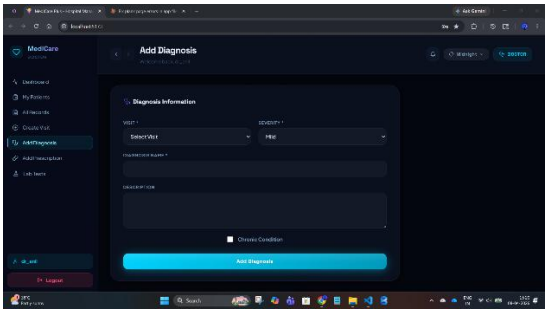


Fig A.7: Add Diagnosis Interface

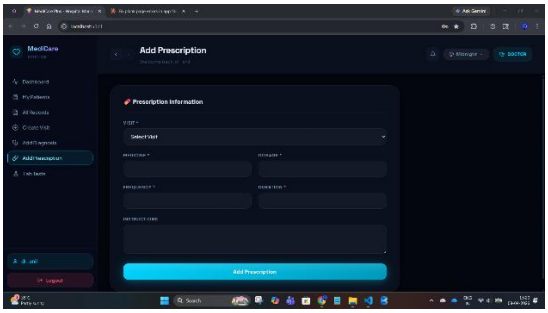


Fig A.8: Add Prescription Interface

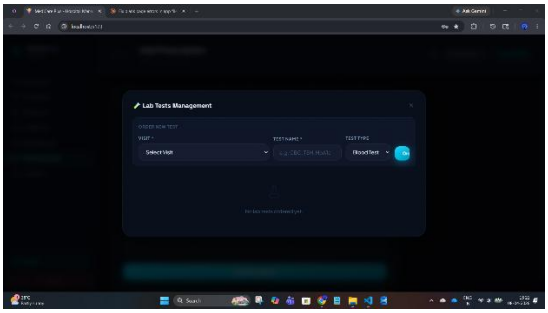


Fig A.9: Lab Tests Order Interface

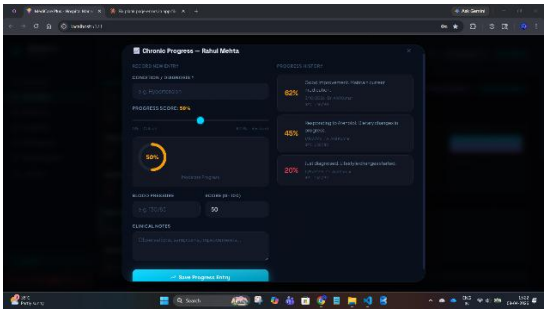


Fig A.10: Chronic Disease Progress Tracking Interface

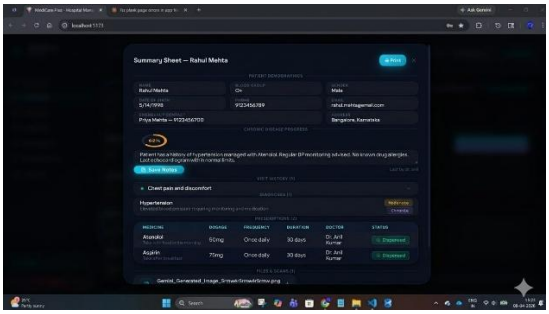


Fig A.11: Patient Summary Sheet Interface

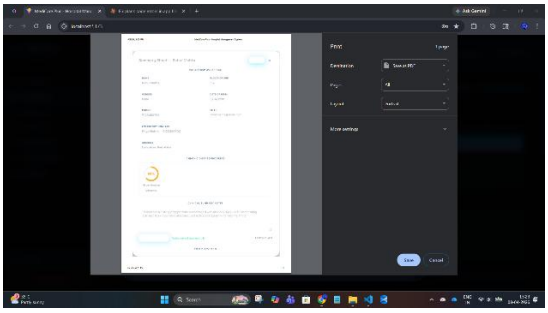


Fig A.12: Report Export Interface

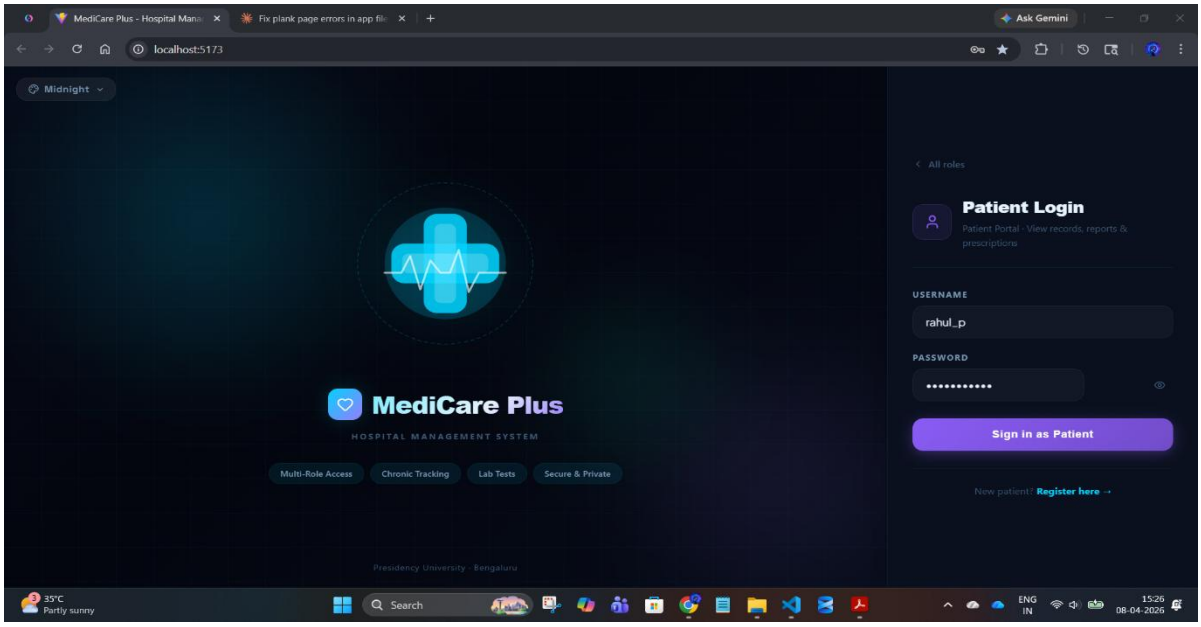
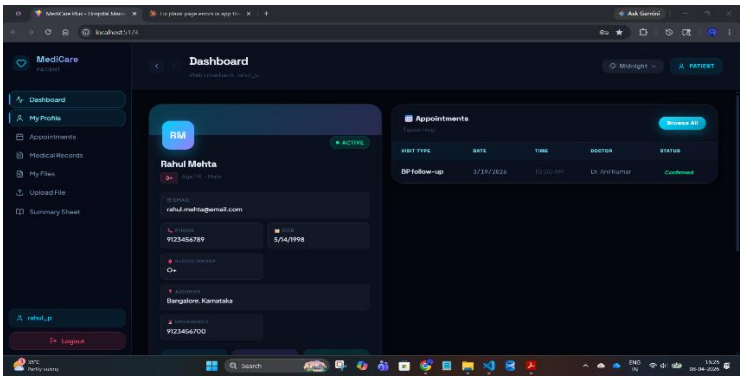


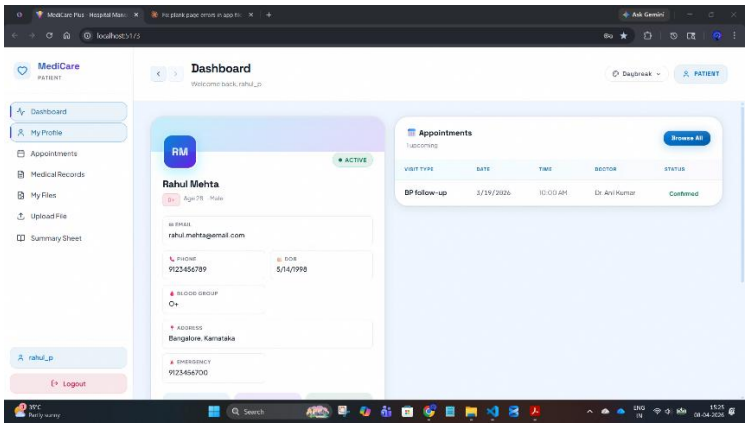
Fig A.13: Patient Login Interface

This image shows the Patient Login screen of MediCare Plus allowing patients to securely access their records, reports, and prescriptions.

PATIENT DASHBOARD:



Dark Theme



Light Theme

Fig A.14: Patient Dashboard Interface

This image shows the Patient Dashboard of MediCare Plus displaying personal details, medical information, and upcoming appointment status.

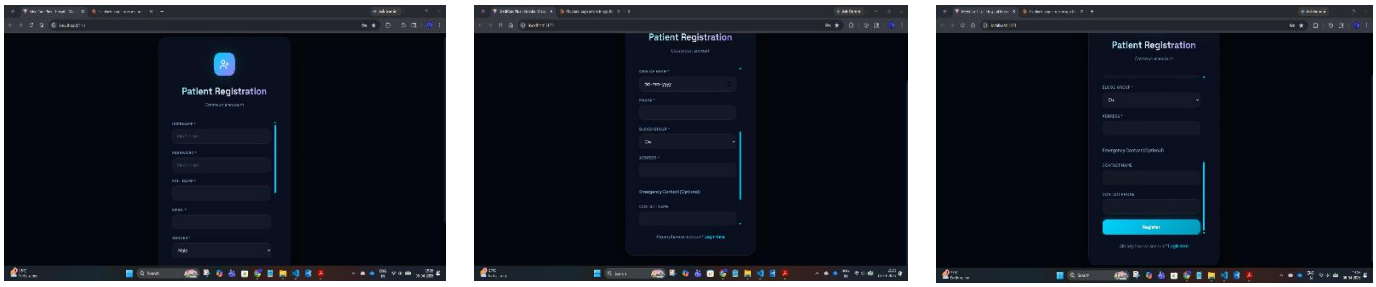


Fig A.15: Patient Registration Interface

This image shows the Patient Registration screen of MediCare Plus where new users enter personal and login details to create an account.

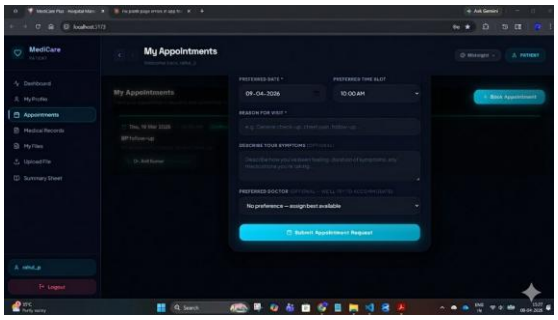


Fig A.16: Appointment Booking Interface

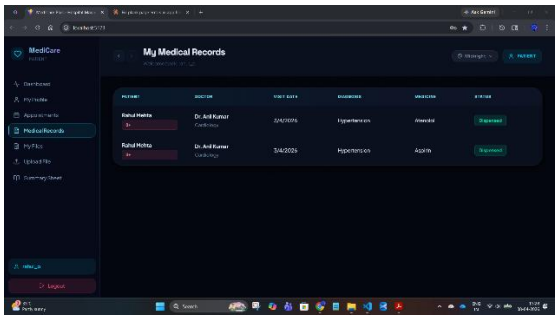


Fig A.17: Patient Medical Records Interface

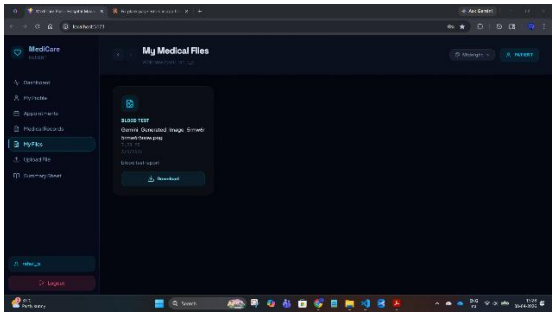


Fig A.18: Medical Files Interface

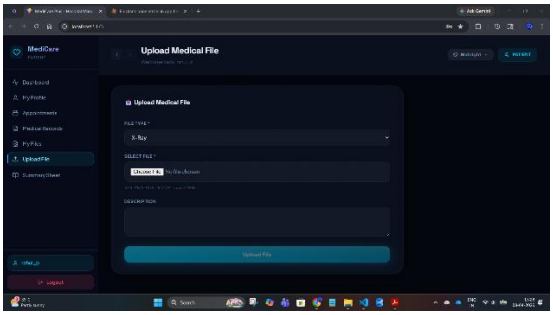


Fig A.19: Upload Medical File Interface

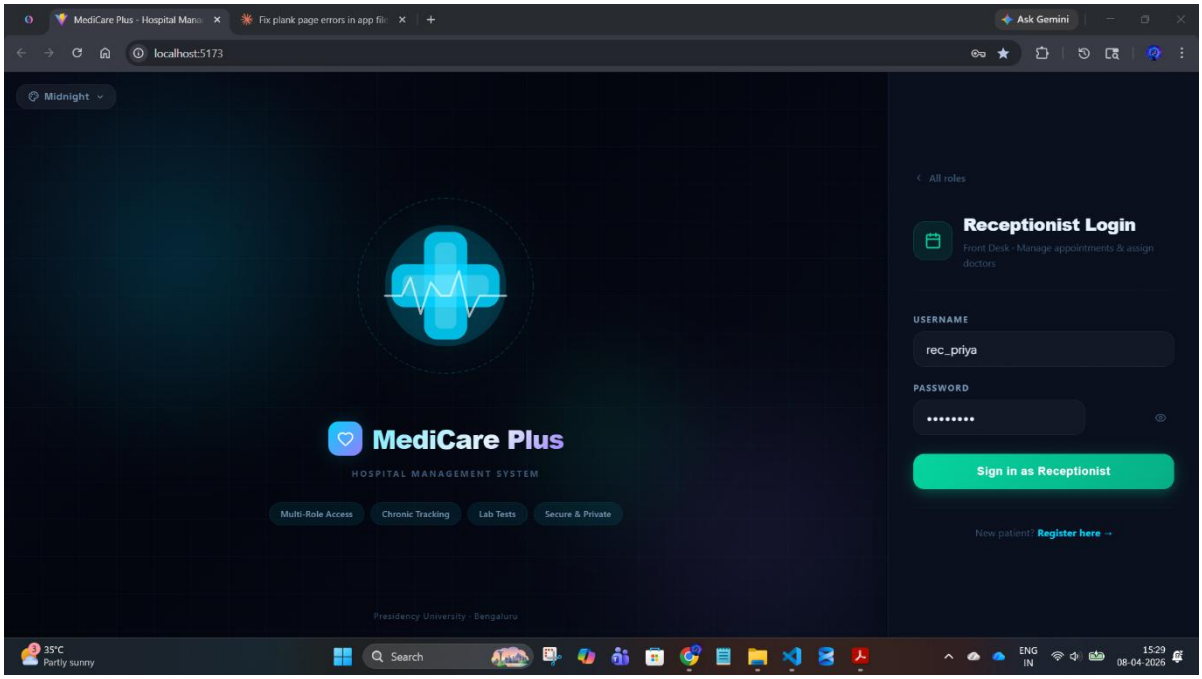


Fig A.20: Receptionist Login Interface

This image shows the Receptionist Login screen of MediCare Plus enabling staff to manage appointments and assign doctors securely.

RECEPTIONIST DASHBOARD:

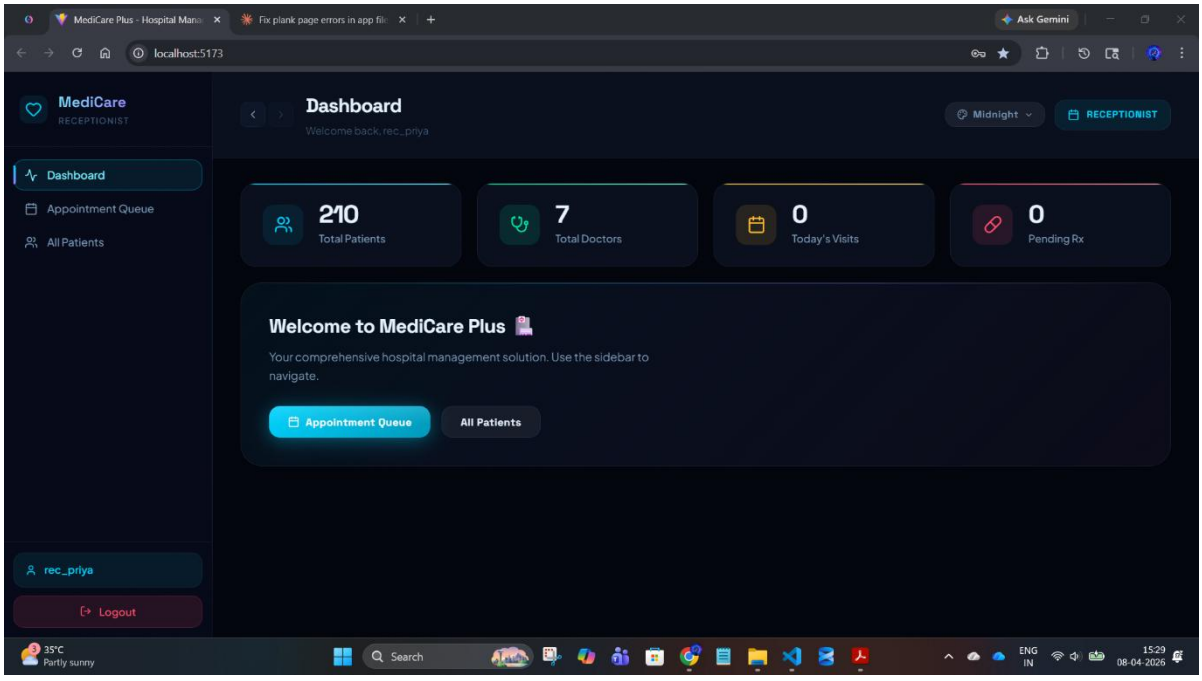


Figure A21: Receptionist Dashboard Interface

This image shows the Receptionist Dashboard of MediCare Plus displaying patient statistics and options to manage appointment queues and patient records.

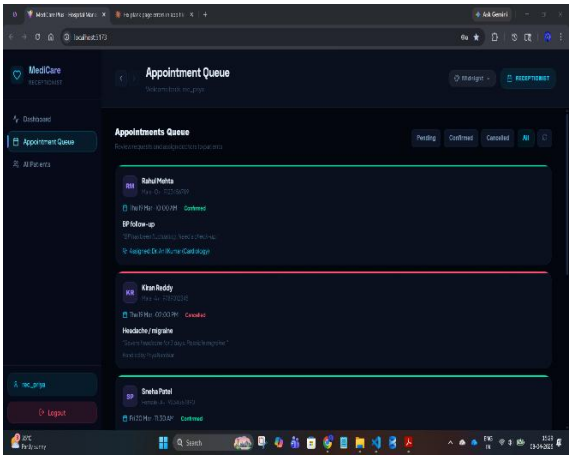


Fig A.22: Appointment Queue Interface

This image shows the Appointment Queue of MediCare Plus where the receptionist reviews, manages, and assigns patient appointments.

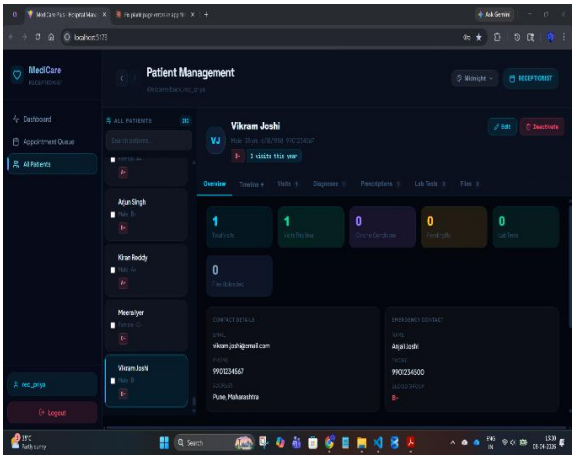


Fig A.23: Patient Management Interface

This image shows the Patient Management screen of MediCare Plus displaying patient details, visit statistics, and record overview.

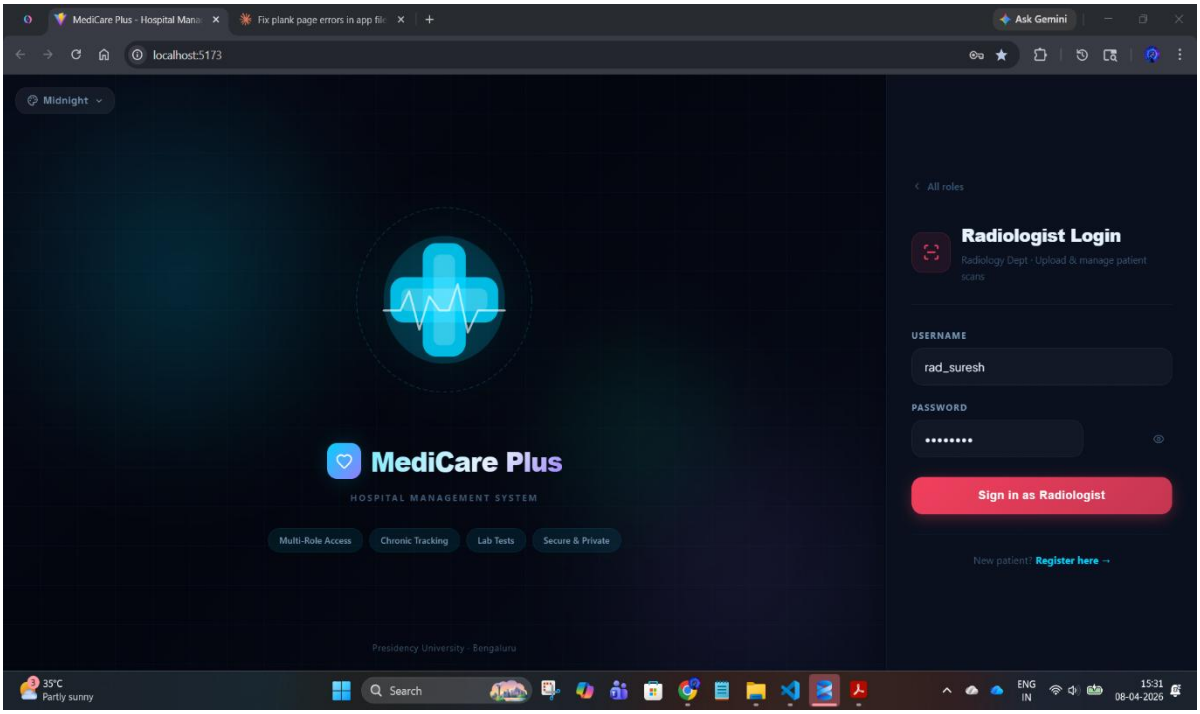


Fig A.24: Radiologist Login Interface

This image shows the Radiologist Login screen of MediCare Plus enabling secure access to upload and manage patient scan reports.

RADIOLOGIST DASHBOARD:

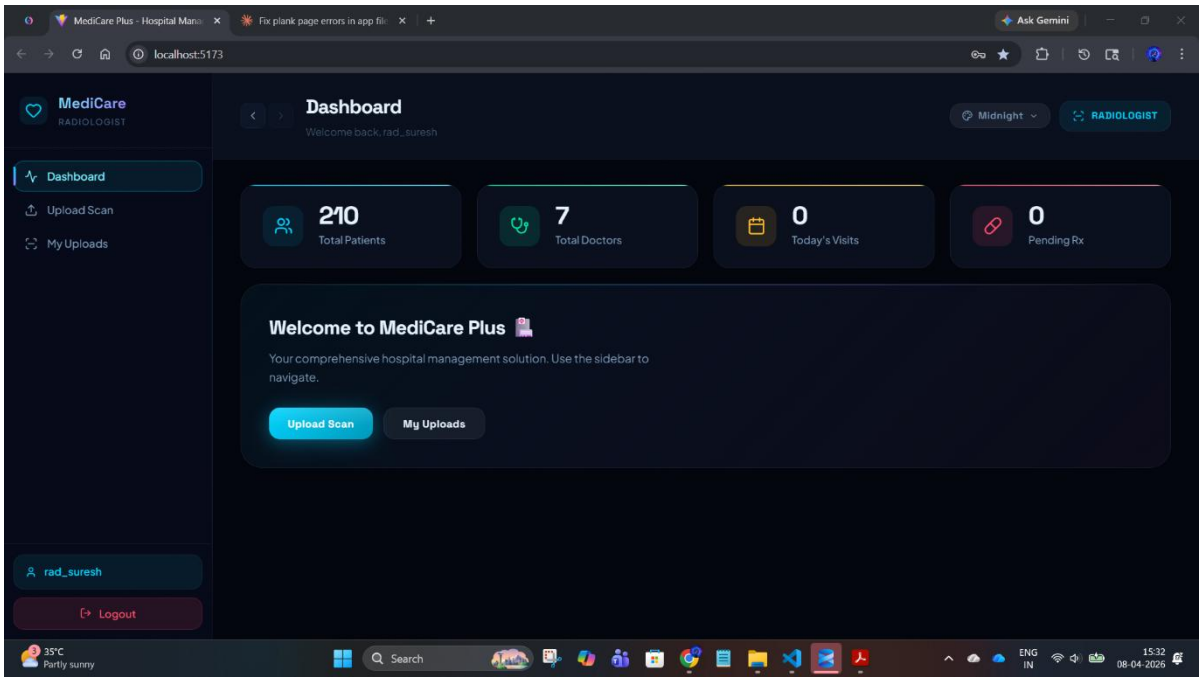


Fig A.25: Radiologist Dashboard Interface

This image shows the Radiologist Dashboard of MediCare Plus displaying system statistics and options to upload and manage diagnostic scans.

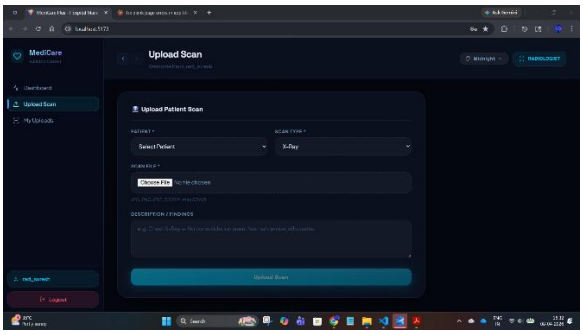


Fig A.26: Upload Scan Interface

This image shows the Upload Scan screen of MediCare Plus where radiologists select patients, upload scan files, and add diagnostic findings.

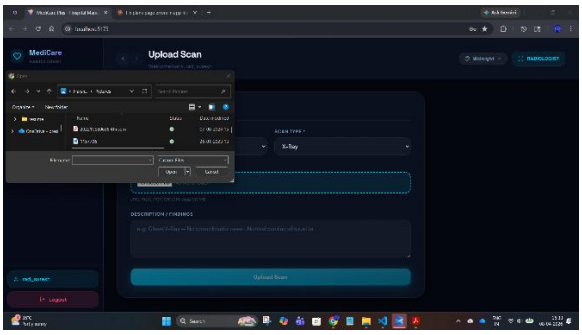


Fig A.27: Scan File Selection Interface

This image shows the file selection dialog used by radiologists to choose and upload scan files from the system.

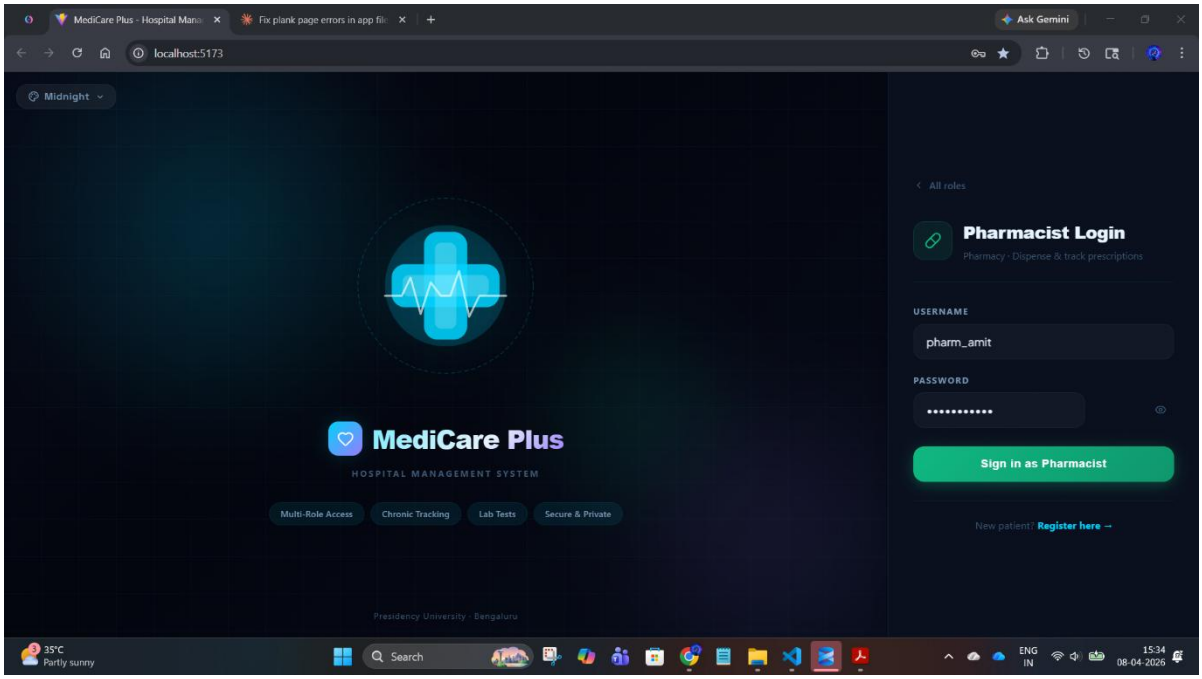


Fig A.28: Pharmacist Login Interface

This image shows the Pharmacist Login screen of MediCare Plus enabling secure access to manage and dispense prescriptions.

PHARMACIST DASHBOARD:

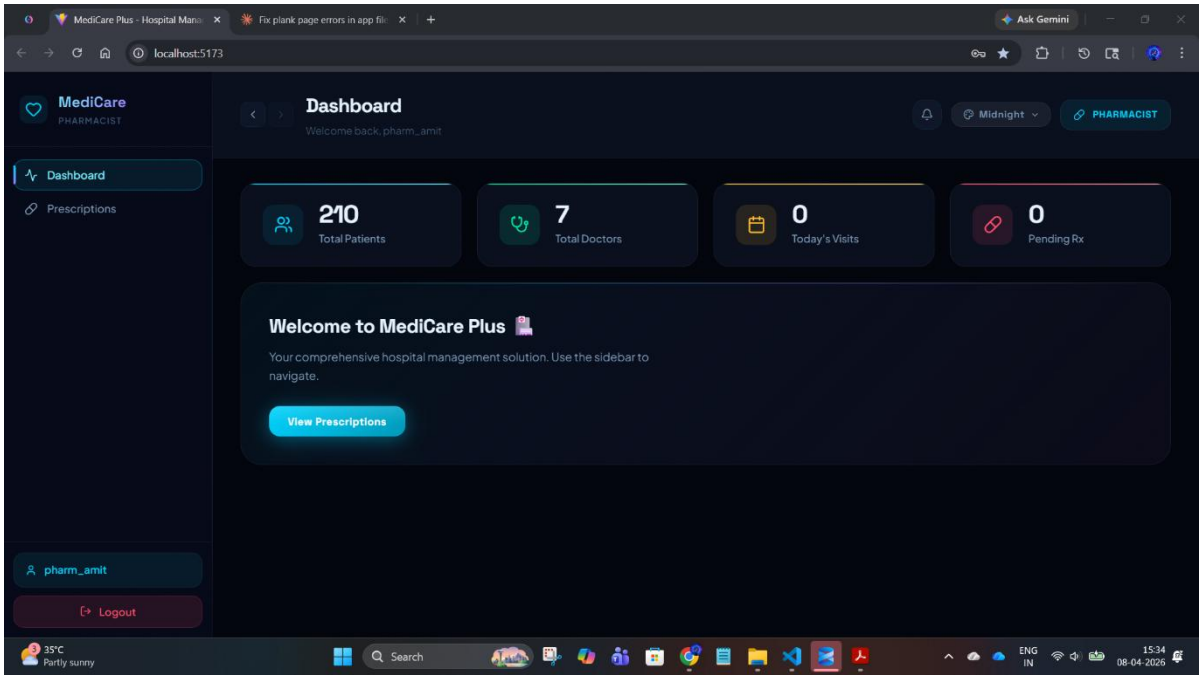


Fig A.29: Pharmacist Dashboard Interface

This image shows the Pharmacist Dashboard of MediCare Plus displaying system statistics and options to view and manage prescriptions.

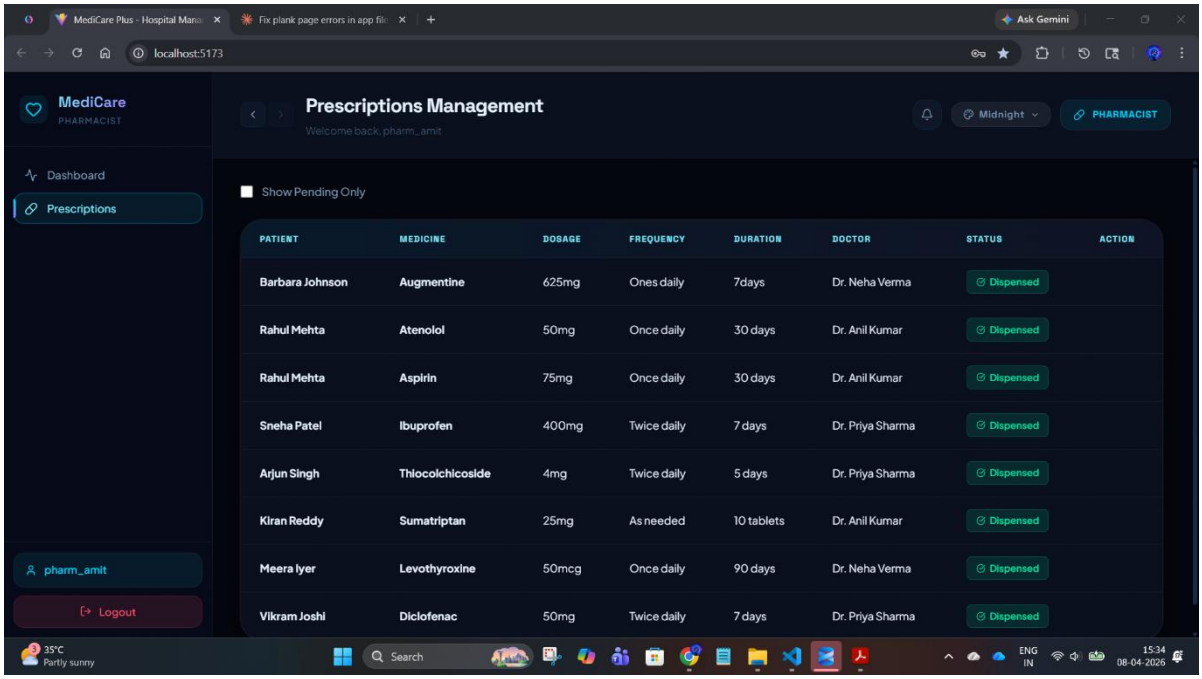


Fig A.30: Prescriptions Management Interface

This image shows the Pharmacist's Prescriptions Management screen displaying patient prescriptions with details such as medicine, dosage, frequency, duration, and dispensing status.

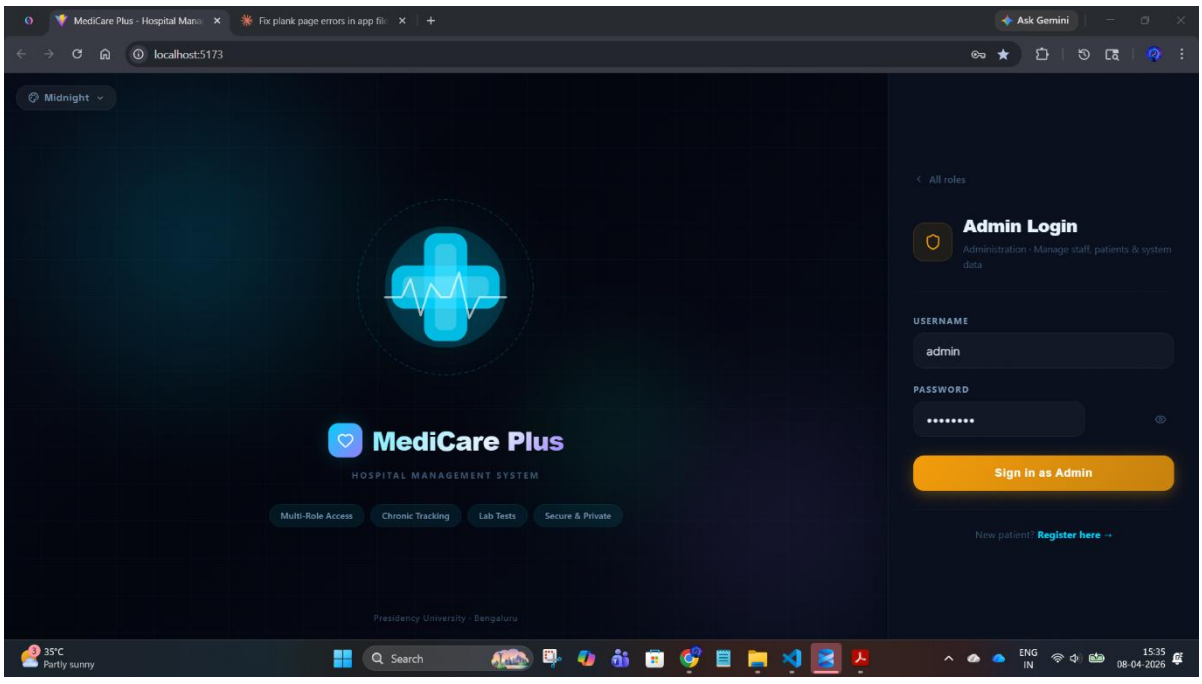


Fig A.31: Admin Login Interface

This image shows the Admin login screen of MediCare Plus with fields for username and password to access system administration features.

ADMIN DASHBOARD:

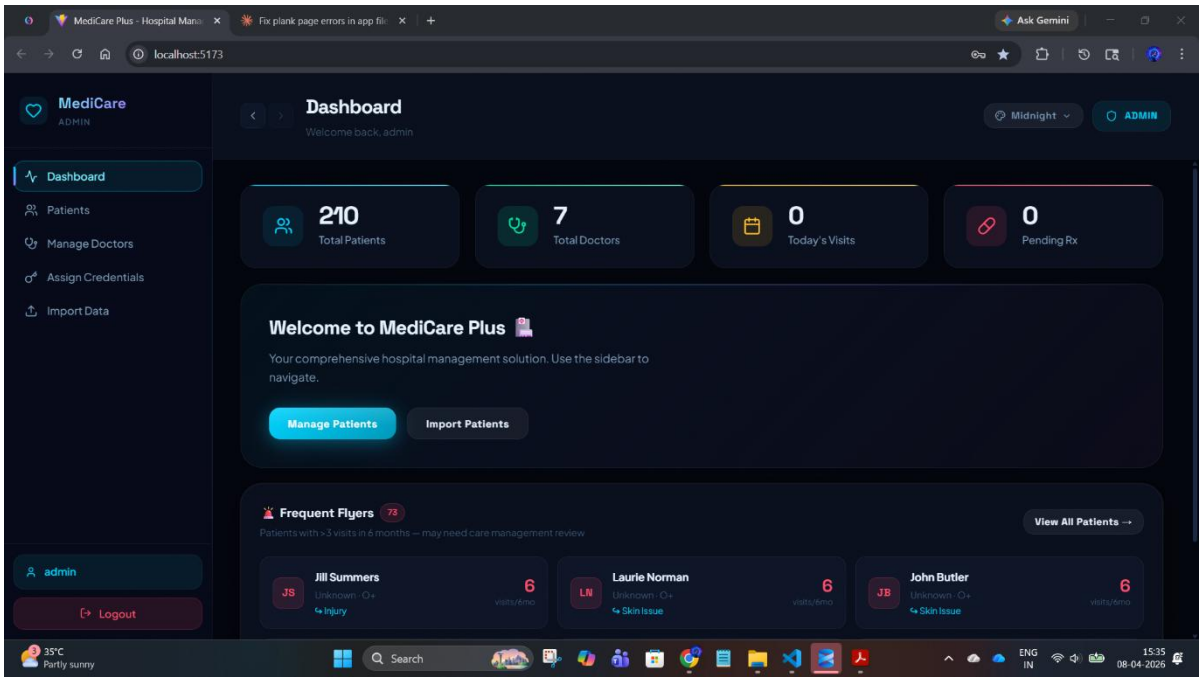


Fig A.32: Admin Dashboard Interface

This image shows the Admin dashboard displaying key metrics like total patients, doctors, visits, and options to manage and import patient data.

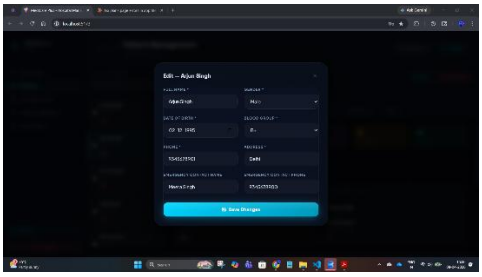


Fig A.33: Edit Patient Details Interface

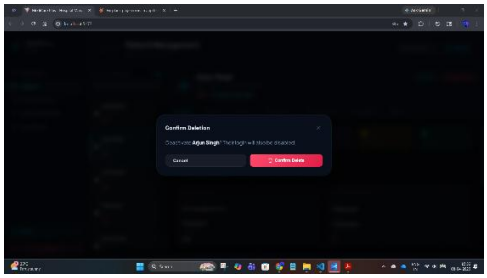


Fig A.34: Delete Confirmation Dialog

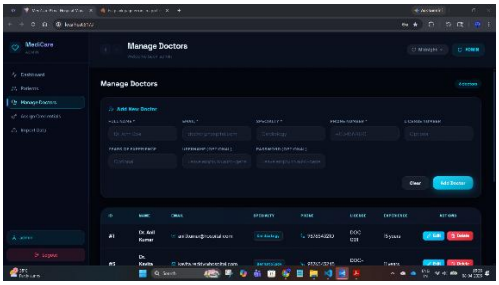


Fig A.35: Manage Doctors Interface

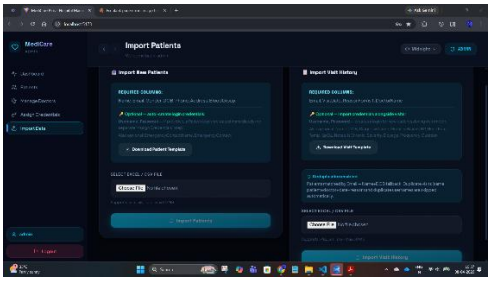


Fig A.36: Import Patients and Visit Data Interface

Appendix - B

Database Schema Summary

Table B.1: Complete Database Table List with Seed Row Counts

Sl. No.	Table Name	Description	Key Columns	Estimated Rows
1	Patients	Stores patient personal and medical details	PatientID (PK), Name, DOB, Gender, Phone	50+
2	Users	Authentication for system users (Admin/Doctor/Staff)	UserID (PK), Username, Password, Role	10+
3	Doctors	Stores doctor profiles and specialization	DoctorID (PK), Name, Specialization, Phone	10+
4	Appointments	Manages patient visit scheduling	AppointmentID (PK), PatientID (FK), DoctorID (FK), Date	100+
5	Visits	Records each hospital visit instance	VisitID (PK), PatientID (FK), VisitDate, Reason	100+
6	Diagnosis	Stores diagnosis details for each visit	DiagnosisID (PK), VisitID (FK), DiagnosisName	80+
7	Medicines	Stores prescribed medicines	MedicineID (PK), Name, Dosage	100+
8	Prescriptions	Maps diagnosis with medicines	PrescriptionID (PK), DiagnosisID (FK), MedicineID (FK)	150+
9	Vitals	Stores patient health metrics	VitalID (PK), BP, HeartRate, Temp, SpO2	100+
10	Departments	Hospital departments (Cardiology, etc.)	DepartmentID (PK), Name	10+

Sl. No.	Table Name	Description	Key Columns	Estimated Rows
11	DoctorDepartment	Mapping doctors to departments	ID (PK), DoctorID (FK), DepartmentID (FK)	15+
12	Billing	Stores billing and payment details	BillID (PK), PatientID (FK), Amount, Status	80+
13	EmergencyContacts	Stores emergency contact details	ContactID (PK), PatientID (FK), Name, Phone	50+
14	MedicalHistory	Tracks chronic conditions	HistoryID (PK), PatientID (FK), IsChronic, Notes	40+
15	Reports	Stores lab/test reports	ReportID (PK), PatientID (FK), ReportType	30+
16	Staff	Hospital staff (non-doctor roles)	StaffID (PK), Name, Role	20+
17	Rooms	Hospital room/ward allocation	RoomID (PK), RoomType, Availability	25+
18	Admissions	Tracks patient admissions	AdmissionID (PK), PatientID (FK), RoomID (FK)	30+
19	Discharges	Tracks patient discharge details	DischargeID (PK), AdmissionID (FK), Date	25+
20	Notifications	System alerts/messages	NotificationID (PK), UserID (FK), Message	100+

Appendix - C

GitHub Repository

Project Source Code Repository

The complete implementation of the **Medicare Plus – Hospital Management System (HMS)** developed during the internship is maintained in a public GitHub repository. The repository includes backend code, frontend modules, database scripts, and supporting documentation.

URL: https://github.com/meharshithv14-ctrl/-Harshith-v-INTERNSHIP

Repository Structure Overview

The repository is organised into multiple folders and files representing different layers of the system architecture.

1. Backend Layer

- **app.py**
  - Main Flask application entry point
  - Handles routing, API initialization, and server execution
- **db.py**
  - Database connection and query handling
  - Implements CRUD operations and SQL integration

2. Frontend Layer

- **hospital-frontend/**
  - Contains user interface files
  - Includes HTML, CSS, and JavaScript
  - Provides dashboards for Admin, Doctor, and Patient

3. Supporting Directories

- **uploads/**
  - Stores uploaded files such as reports and medical documents
- **Scripts/**
  - Contains environment or execution-related scripts
- **Lib/site-packages/**
  - Python dependencies and installed libraries (virtual environment)

4. Documentation Files

- **README.md**
  - Overview of the project, features, and setup instructions
- **Algorithm Details.docx**
  - Explanation of implemented logic and workflows
- **Source Code Details.docx**
  - Detailed breakdown of modules and code structure
- **Entire project ss.docx**
  - Screenshots of the system interfaces
- **Medicare Plus report**
  - Final internship report document
- **hms_code.docx**
  - Additional source-level explanations

5. Configuration Files

- **.gitignore**
  - Specifies files and folders excluded from version control